



Case Study: Simulation of CPU Scheduling Algorithm

**A Case Study Presented to
The Faculty of the Bachelor of Science in Computer Science
College of Computer Studies**

**In Partial Fulfillment
of the Requirements for the Course
Operating System**

Prepared by:

Ruales, Rycel Joshua M.

BSCS - 3B

Submitted to:

Mrs. Joanne Cura

Due Date: May 15, 2026



I. Introduction

Introduction to CPU Scheduling

CPU scheduling is a key operating system concept that deals with the scheduling of processes on the CPU. Most systems now execute multiple processes but only has a single CPU (or a small number of CPUs), so the operating system must determine which process to run at any given time. This is the role of the CPU scheduler, which is part of the operating system kernel (Ali et al., 2021).

Scheduling directly impacts system performance metrics such as throughput, response time, waiting time, and overall fairness (Stallings, 2018). An inefficient scheduler can lead to processes not running for extended periods known as starvation or a system being perceived as sluggish. In contrast, an efficient scheduler ensures high CPU utilization and a responsive system.

CPU scheduling algorithms are broadly categorized into two types which are non-preemptive and preemptive. Under non-preemptive scheduling, a process will run until it finishes or explicitly relinquishes the CPU, such as during input/output (I/O) operations. This type of scheduling includes algorithms such as First-Come, First-Served (FCFS), Shortest Job First (SJF), and non-preemptive Priority scheduling. In contrast, in preemptive scheduling, the operating system can pre-empt a process and allocate the CPU to another process, often based on priority or time-sharing. Examples of this type include Shortest Remaining Time (SRT), Round Robin, and Priority with Round Robin scheduling (Chakraborty et al., 2024).

Purpose of Case Study

The primary objective of this case study is to offer a practical approach to learning about six commonly investigated CPU scheduling algorithms. It focuses not only on the theoretical aspects of the algorithms but also on the implementation and execution of a scheduling simulator to observe their behavior in practice. As Ali et al. (2021) noted, this allows the study of the operation of each scheduling algorithm under different scenarios, such as varying loads, arrival rates, and CPU burst times.

By simulating these environments, it enables a more thorough examination of the impact of scheduling algorithms on performance measures like waiting time, turnaround time and system throughput. It also showcases the effectiveness and shortcomings of each algorithm under varying conditions. This comparative analysis also offers a better understanding of the relative strengths of different algorithms under various conditions and the trade-offs between fairness, responsiveness and efficiency (Chakraborty et al., 2024).



Objectives of the Program

The goal of this case study is to implement and test several standard CPU scheduling algorithms in a practical manner by designing a simulation model. Through the simulation of theoretical scheduling algorithms, this study seeks to enhance understanding of their behavior in a practical environment.

In particular, this study aims to:

- Develop a simulation framework incorporating six classical CPU scheduling algorithms, both preemptive and non-preemptive, in a single platform.
- Execute processes by providing a range of input scenarios including arrival times, burst times, and priority levels, to simulate different workload scenarios.
- Observe and compare performance metrics such as waiting time, turnaround time and CPU utilization to assess the effectiveness of various algorithms.
- Analyze the behavior and performance of the algorithms in order to determine their advantages and limitations with regards to responsiveness, fairness and efficiency in resource utilization.
- Analyze the impact of various scheduling policies on the system's performance under certain conditions, such as increased process load or varying arrival rates.
- Offer a structured analysis of the results using tables and Gantt charts to draw conclusions.

Through these objectives, this study seeks to enhance conceptual knowledge and skills in CPU scheduling, as well as to provide guidance on the choice of suitable scheduling methods for various system needs.



II. Body

1. First-Come, First-Served (FCFS)

Algorithm Description and Type: Non-Preemptive

The First-Come, First-Served (FCFS) algorithm is the most basic CPU scheduling algorithm. This is a First-In, First-Out (FIFO) scheduling algorithm, where processes are run in the same order they enter the ready queue. Processes are executed without preemption, meaning a process will run until it completes. It doesn't consider burst time or priority, so processes are executed in a sequence determined by arrival time (Ali et al., 2021). Within the simulation, the first step is to order the processes according to their arrival time. If there's no process waiting to run, the CPU is idle, and in the Gantt chart, this is represented as an idle time to maintain the timeline of the Gantt chart.

Advantages and Limitations

Advantages:

The FCFS scheduling algorithm is straightforward and easy to understand, with the only required operations being basic queue operations with little decision-making involved. It guarantees every process will be processed, thus avoiding starvation. It also has a deterministic execution order based solely on the processes' arrival times, resulting in easier traceability, debugging, and analysis of the system during simulation and testing. It is also a simple scheduling algorithm with low overhead, as it does not involve complex calculations or frequent context switching.

Limitations:

While FCFS is simple, it has several limitations. It is prone to the convoy effect, where a long process at the head of the queue holds up several shorter processes, resulting in poor CPU utilization. It does not consider other critical parameters like process priority or process burst time and can therefore not be used for interactive systems or real-time systems, where responsiveness is required. Also, the average waiting time may be much longer if there's significant variability in the burst times, which will degrade system performance and user experience.



FCFS – Test Case 1

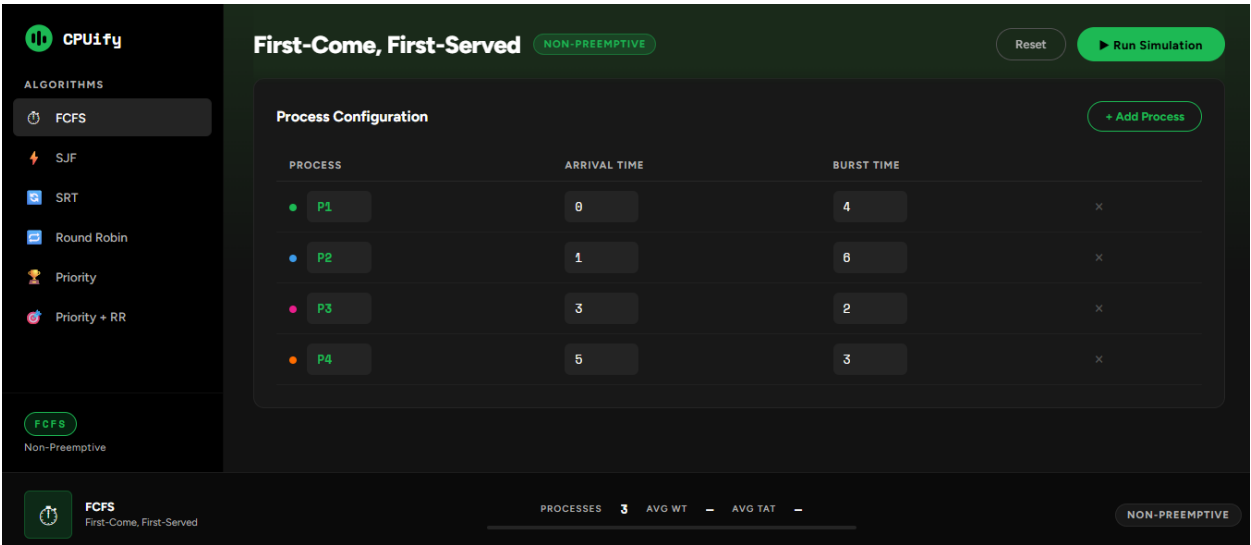


Figure 1. Interface of the System - FCFS

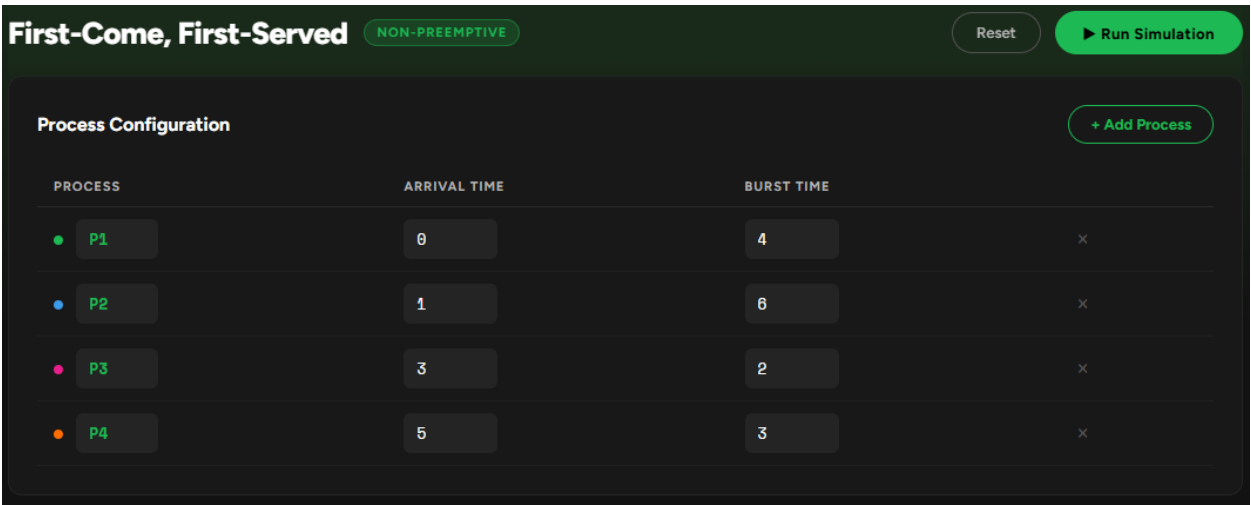


Figure 2. Input Parameters of FCFS Test Case 1

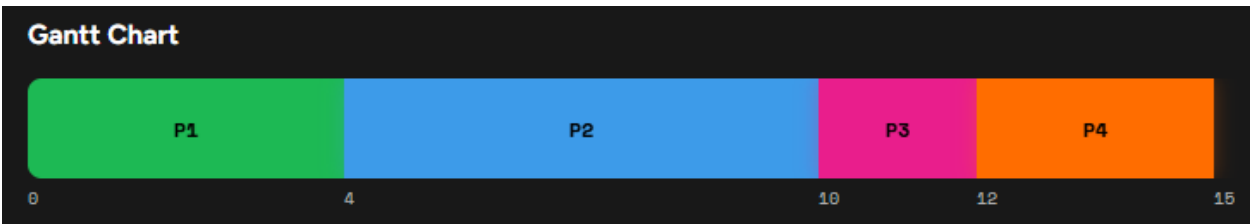
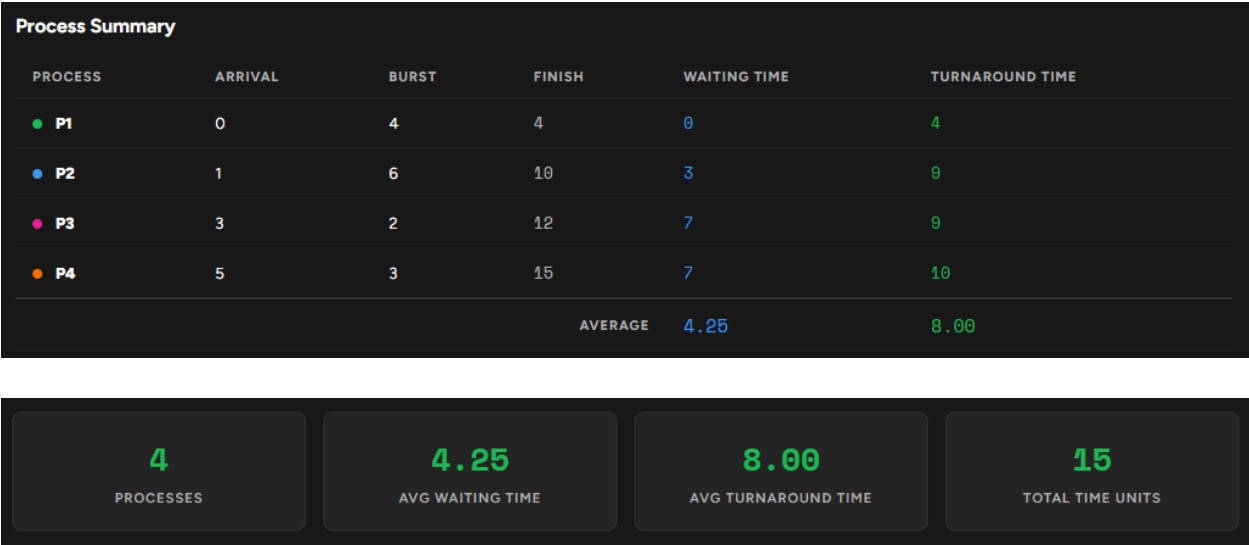


Figure 3. Gantt Chart of FCFS Test Case 1



Figures 4 & 5. Process Summary of FCFS Test Case 1

At time 0, only P1 has arrived, so it immediately takes control of the CPU and executes continuously for its full burst of 4-time units, completing at time t=4. When the CPU becomes available at t=4, the remaining processes (P2, P3, and P4), which have already arrived, are queued in the ready line. Following the FCFS rule, execution proceeds based on arrival order: P2 (arrived at t=1) runs first from t=4 to t=10, followed by P3 (arrived at t=3) from t=10 to t=12, and finally P4 (arrived at t=5) from t=12 to t=15. Although P3 and P4 have relatively short burst times of 2 and 3 units, they are delayed because they are positioned behind the longer-running P2, clearly illustrating the convoy effect in FCFS scheduling.

FCFS – Test Case 2

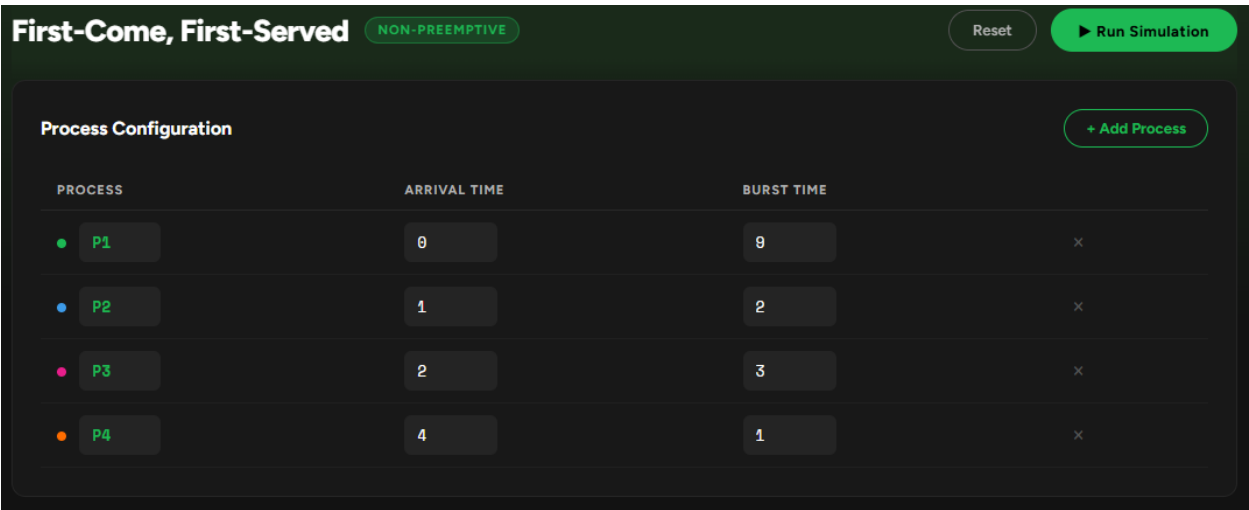


Figure 6. Input Parameters of FCFS Test Case 2

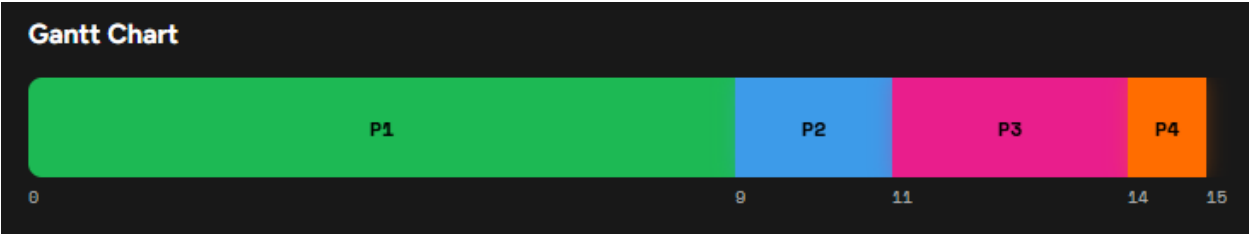
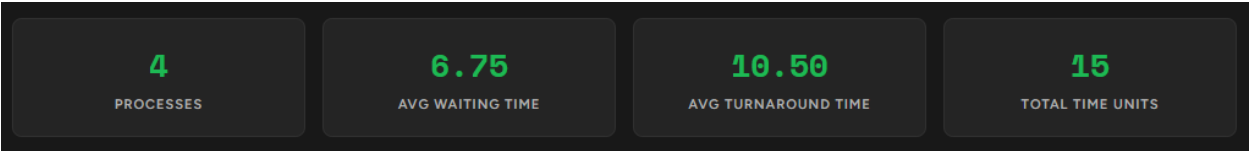


Figure 7. Gantt Chart of FCFS Test Case 2

Process Summary					
PROCESS	ARRIVAL	BURST	FINISH	WAITING TIME	TURNAROUND TIME
P1	0	9	9	0	9
P2	1	2	11	8	10
P3	2	3	14	9	12
P4	4	1	15	10	11
AVERAGE				6.75	10.50



Figures 8 & 9. Process Summary of FCFS Test Case 2

This test case is intentionally designed to highlight the Convoy Effect. P1 arrives first and occupies the CPU for a long duration of 9-time units. During this period, P2 (BT=2), P3 (BT=3), and P4 (BT=1) arrive and line up in the ready queue, all forced to wait behind the single long-running process. Once P1 finishes, execution continues in arrival order, P2 runs from t=9 to t=11, followed by P3 from t=11 to t=14, and finally P4 from t=14 to t=15. Despite having the shortest burst time, P4 experiences a long waiting time of 10 units simply because it arrived behind a long process. This results in a high average waiting time of 6.75, demonstrating how FCFS can severely degrade performance when long burst processes are scheduled early.

2. Shortest Job First (SJF)

Algorithm Description and Type: Non-Preemptive

Shortest Job First (SJF) scheduling policy examines all processes that have already arrived and are ready to be executed (waiting in the ready queue) when the CPU becomes free. It then chooses the process with the shortest burst time and runs it until it completes. In case of ties, arrival time is used to ensure a predictable ordering of processes (Omar et al., 2021).



Advantages and Limitations

Advantages:

Shortest Job First (SJF) is mathematically proven to produce the lowest average waiting time among all non-preemptive scheduling algorithms for a given set of processes. It effectively reduces the convoy effect by prioritizing shorter processes whenever they are available in the ready queue, allowing quicker completion of small tasks and improving overall system efficiency. In addition, once burst times are known or estimated, the scheduling decision process becomes straightforward and easy to implement.

Limitations:

SJF relies on having accurate knowledge of each process’s burst time, which is rarely available in real operating systems without prediction or estimation techniques (Ali et al., 2021). It also introduces the risk of starvation, where long processes may be repeatedly delayed if shorter processes continue to arrive. Furthermore, SJF is not ideal for interactive or time-sharing systems, as it prioritizes overall efficiency rather than providing fast response times for individual user requests.

SJF – Test Case 1

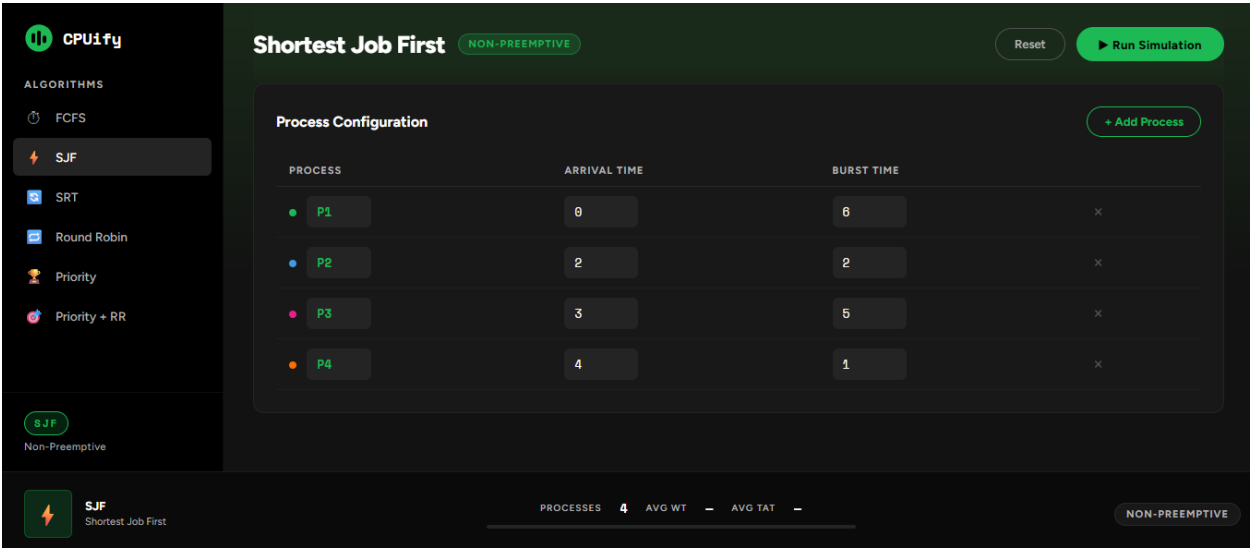


Figure 10. Interface of the System – SJF

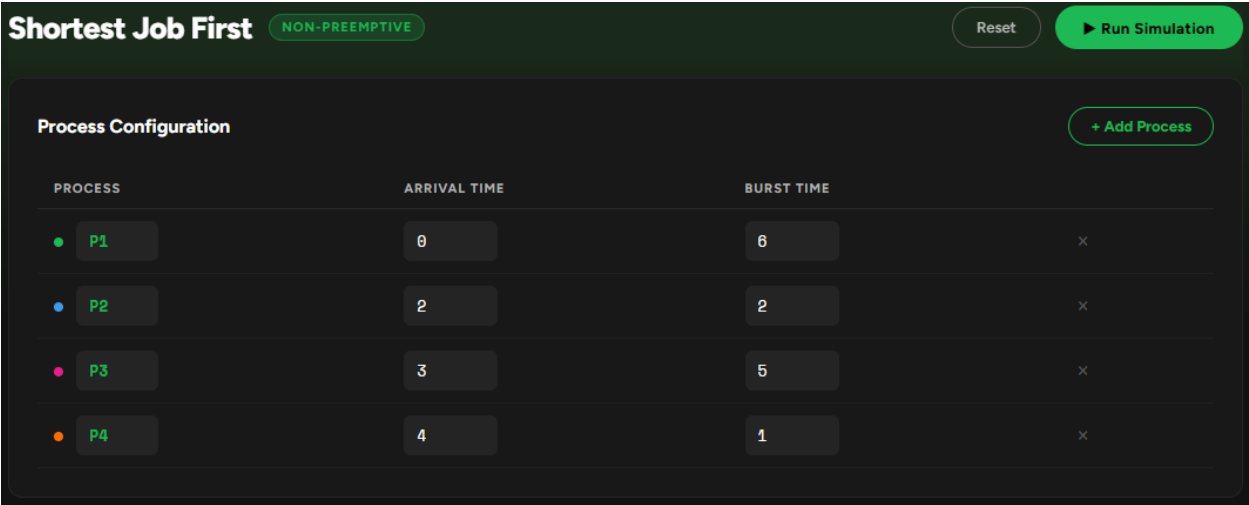
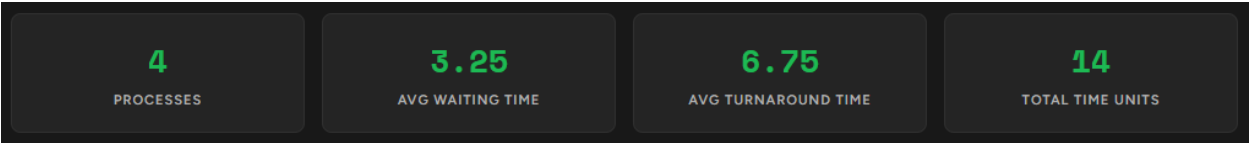


Figure 11. Input Parameters of SJF Test Case 1



Figure 12. Gantt Chart of SJF Test Case 1

Process Summary					
PROCESS	ARRIVAL	BURST	FINISH	WAITING TIME	TURNAROUND TIME
P1	0	6	6	0	6
P2	2	2	9	5	7
P3	3	5	14	8	11
P4	4	1	7	2	3
AVERAGE				3.25	6.75



Figures 13 & 14. Process Summary of SJF Test Case 1

At time $t=0$, only P1 is available, so it immediately executes for 6-time units, finishing at $t=6$. Once the CPU becomes available, all remaining processes (P2 with $BT=2$, P3 with $BT=5$, and P4 with $BT=1$) are already in the ready queue. Following the SJF rule, the scheduler selects the shortest job first, so P4 executes from $t=6$ to $t=7$. Next, P2 runs from $t=7$ to $t=9$ due to its relatively small burst time, and finally P3 is executed from $t=9$ to $t=14$ as the longest remaining process. By prioritizing shorter processes before longer ones, SJF significantly improves average waiting time compared to FCFS, where execution is strictly based on arrival order.



SJF – Test Case 2

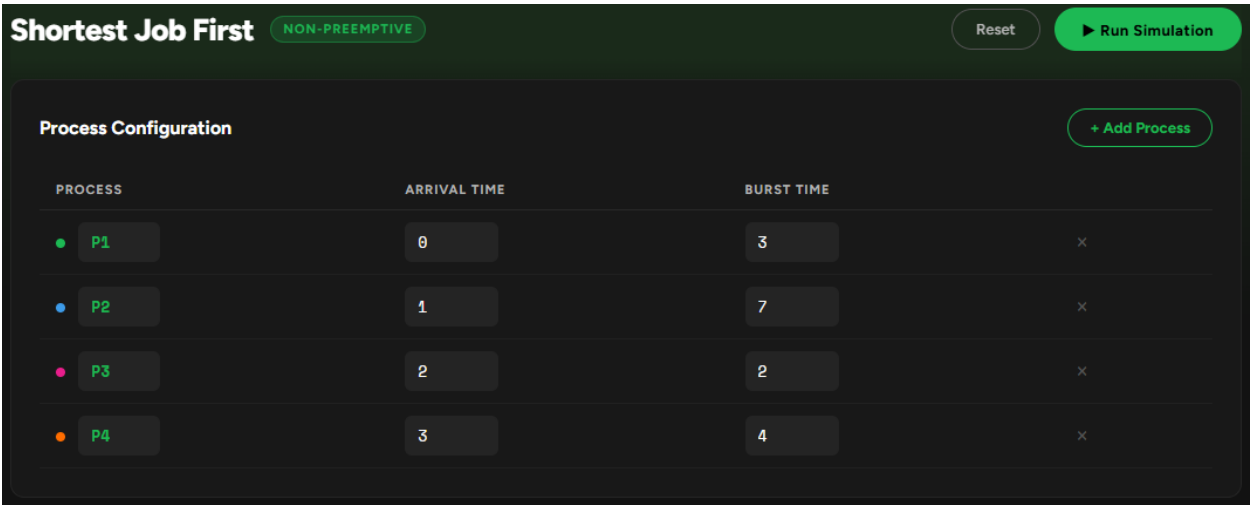


Figure 15. Input Parameters of SJF Test Case 2

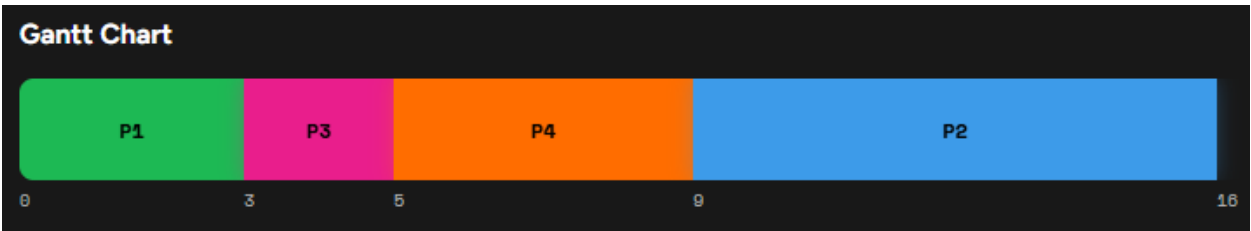
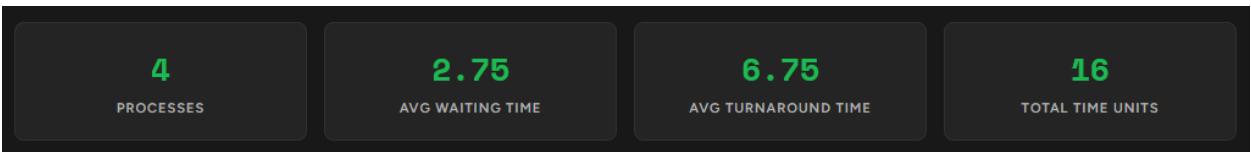


Figure 16. Gantt Chart of SJF Test Case 2

Process Summary					
PROCESS	ARRIVAL	BURST	FINISH	WAITING TIME	TURNAROUND TIME
P1	0	3	3	0	3
P2	1	7	16	8	15
P3	2	2	5	1	3
P4	3	4	9	2	6
AVERAGE				2.75	6.75



Figures 17 & 18. Process Summary of SJF Test Case 2

At time $t=0$, P1 is the only available process and immediately executes from $t=0$ to $t=3$. By the time the CPU becomes free at $t=3$, P2 (BT=7), P3 (BT=2), and P4 (BT=4) are all waiting in the ready queue. Following the SJF rule, the scheduler selects the shortest process first, so P3 executes from $t=3$ to $t=5$. Next, P4 runs from $t=5$ to $t=9$ due to its moderate burst time, and finally P2 executes from $t=9$ to $t=16$ as the longest remaining job. Although P2 arrived early, it is delayed



significantly, waiting a total of 8-time units because shorter processes arriving later were prioritized. This clearly illustrates SJF's vulnerability to starvation in dynamic arrival scenarios involving long-burst processes.

3. Shortest Remaining Time (SRT)

Algorithm Description and Type: Preemptive

Shortest Remaining Time (SRT) is the preemptive version of Shortest Job First. Rather than making a scheduling decision only when the CPU is idle, SRT keeps track of the remaining burst times of all processes that have entered the system. If at any time a new process arrives with a remaining burst time smaller than that of the process currently running, the CPU is preempted and the currently running process is reinserted in the queue, while the new process runs (Ali et al., 2021).

Advantages and Limitations

Advantages:

Shortest Remaining Time (SRT) is one of the most efficient scheduling algorithms in terms of average waiting time for preemptive scheduling policies and is optimal for most workloads in terms of CPU usage. It is also very responsive to the arrival of new, short processes, as these will be prioritized and scheduled for execution as soon as they enter the ready queue. Moreover, SRT efficiently uses the CPU by always giving the CPU to the process that's closest to finishing, enhancing system throughput.

Limitations:

While efficient, SRT involves frequent context switching as it continually preempts processes, adding to the overhead and potentially limiting the performance gain. It is also prone to starvation, particularly for processes with long burst times, which can be exacerbated by the arrival of new short processes. Additionally, the algorithm relies heavily on either the exact or relatively close estimated burst time of the processes, as inaccurate estimates will lead to poorer scheduling decisions.



SRT – Test Case 1

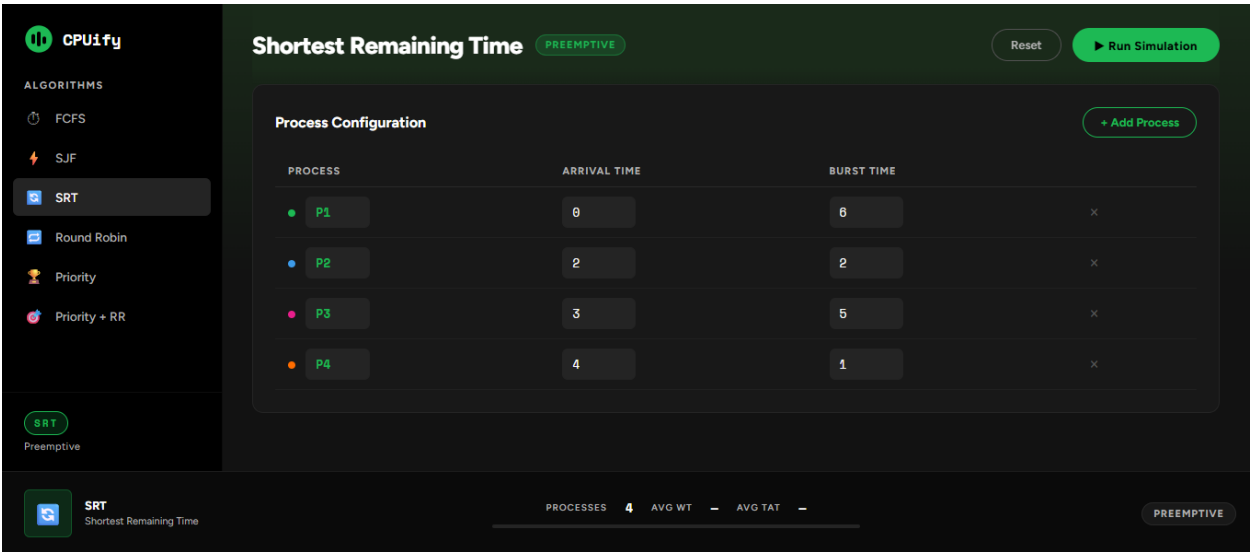


Figure 19. Interface of the System - SRT

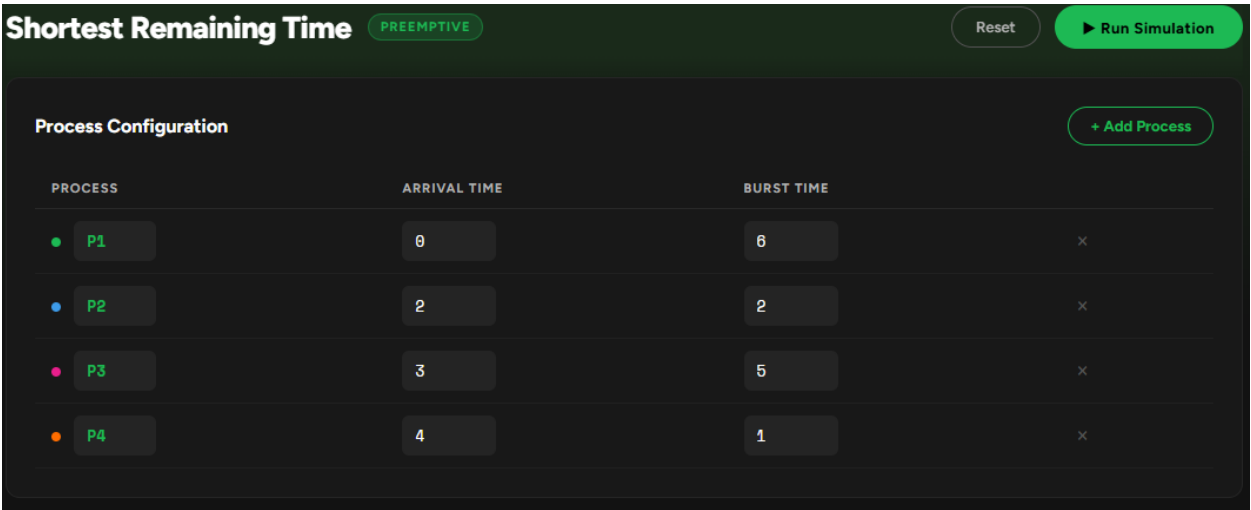


Figure 20. Input Parameters of SRT Test Case 1

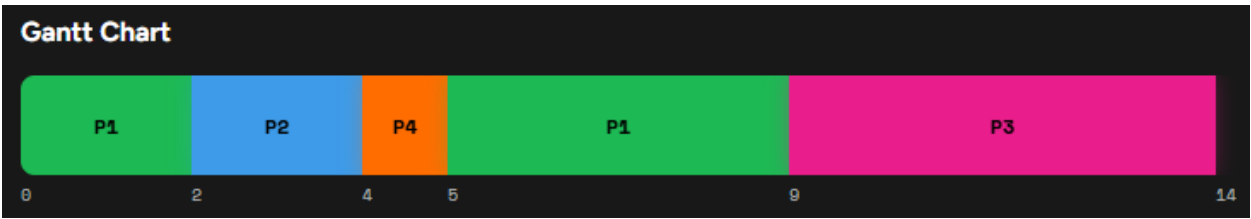
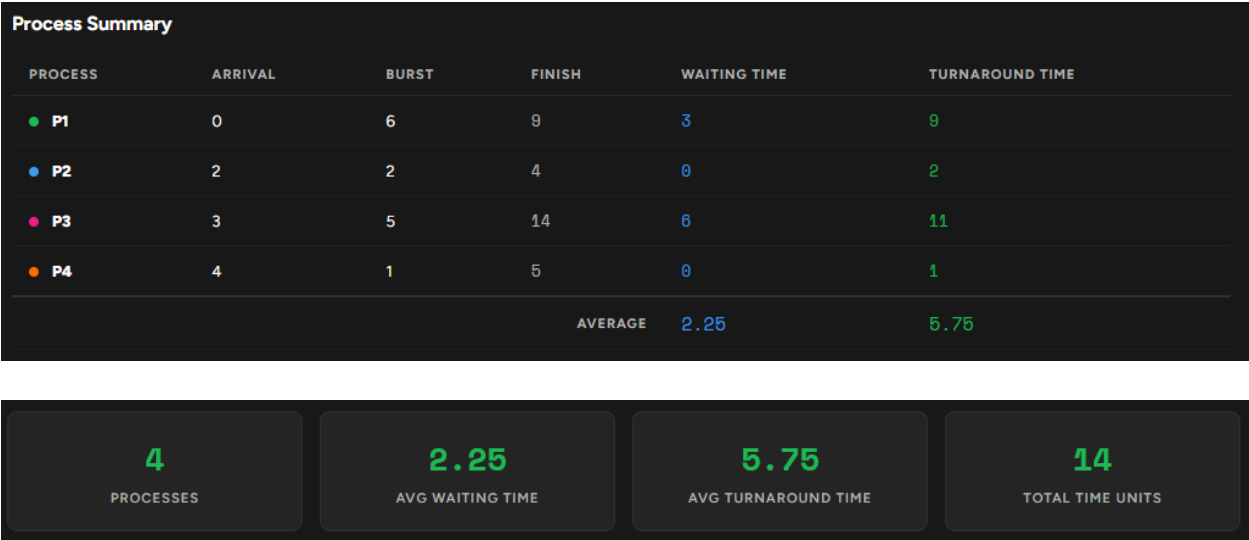


Figure 21. Gantt Chart of SRT Test Case 1



Figures 22 & 23. Process Summary of SRT Test Case 1

This test case uses the same set of processes as SJF Test Case 1, enabling a direct comparison between the two algorithms. Execution begins with P1 at t=0 with a remaining time of 6. At t=2, P2 arrives with a burst time of 2, which is shorter than P1’s remaining 4 units, causing P1 to be preempted and P2 to take over. At t=3, P3 arrives with a burst time of 5, which is greater than P2’s remaining 1 unit, so no preemption occurs. At t=4, P4 arrives with a burst time equal to P2’s remaining time (1), and since there is no strict advantage, P2 continues and completes at t=4. Afterward, P4, having the shortest burst time, executes from t=4 to t=5. Among the remaining processes, P1 (remaining 4) is selected over P3 (burst 5) and runs from t=5 to t=9. Finally, P3 executes from t=9 to t=14. This results in a lower average waiting time compared to SJF, as shorter processes like P2 and P4 are served immediately upon arrival due to preemption.

SRT – Test Case 2

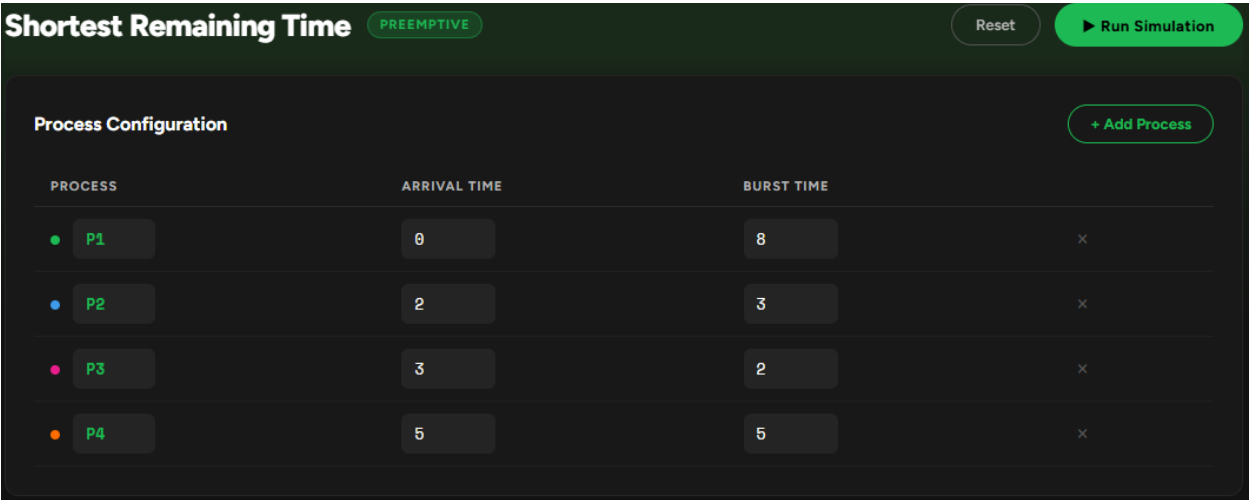


Figure 24. Input Parameters of SRT Test Case 2

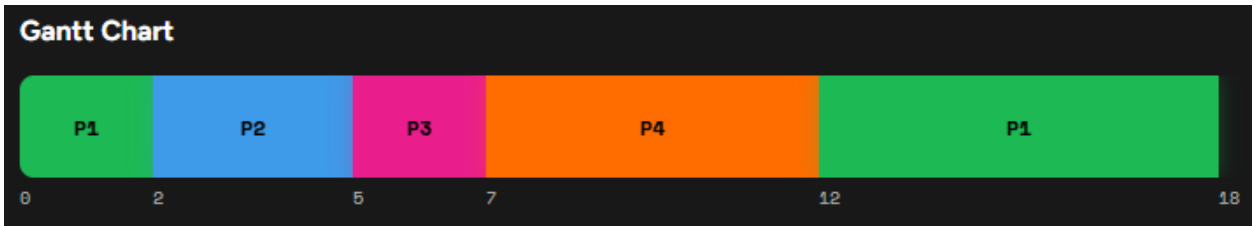
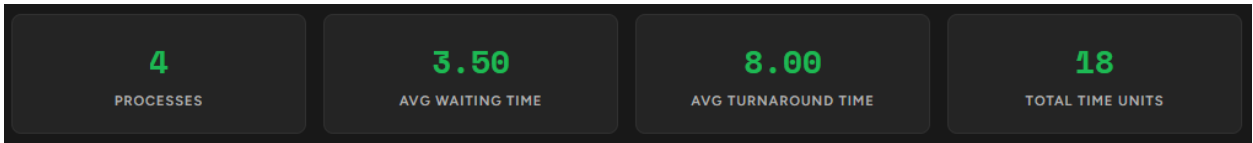


Figure 25. Gantt Chart of SRT Test Case 2

Process Summary					
PROCESS	ARRIVAL	BURST	FINISH	WAITING TIME	TURNAROUND TIME
P1	0	8	18	10	18
P2	2	3	5	0	3
P3	3	2	7	2	4
P4	5	5	12	2	7
AVERAGE				3.50	8.00



Figures 26 & 27. Process Summary of SRT Test Case 2

Execution begins with P1 at $t=0$, which starts running with a remaining time of 8. At $t=2$, P2 arrives with a burst time of 3, which is shorter than P1’s remaining 6 units, so P1 is preempted and P2 takes over. At $t=3$, P3 arrives with a burst time equal to P2’s remaining time (2), and since there is no strict advantage, P2 continues execution and completes at $t=5$. At this point, P3 (remaining 2), P4 (burst 5), and P1 (remaining 6) are all ready, and P3 is selected as it has the shortest remaining time, finishing at $t=7$. Next, P4 (remaining 5) is chosen over P1 (remaining 6) and runs from $t=7$ to $t=12$. Finally, P1 resumes execution and completes from $t=12$ to $t=18$. This sequence illustrates how longer processes accumulate significant waiting time, as shorter processes are consistently prioritized and executed first.

4. Round Robin (RR)

Algorithm Description and Type: Preemptive and Requires Time Quantum

The Round Robin scheduling algorithm allocates a time slice (also called time quantum) to each process and processes them in a circular order in the ready queue. The process at the head of the queue is given the CPU for a maximum of one quantum (time slice); if it does not finish within that time, it is preempted and moved to the end of the queue for the next round of scheduling,



giving the next process a chance to execute. If a process completes before its quantum, the CPU is promptly given to the next process (Omar et al., 2021).

Advantages and Limitations

Advantages:

Round Robin is one of the fairest CPU scheduling algorithms as it allocates a fair share of the CPU to each process. The algorithm prevents starvation, ensuring that no process is indefinitely postponed from execution, since each process gets periodic access to the CPU within a set cycle. In this respect, Round Robin is ideal for time-sharing systems and interactive computing, where responsiveness is critical and total waiting time is of less concern.

Limitations:

Although Round Robin is fair, its performance varies with the size of the quantum. A short quantum leads to high context switching, which imposes a high overhead on the system and can decrease the CPU efficiency. Conversely, a large quantum results in an algorithm that behaves like FCFS, with poor responsiveness, and therefore slows down the fairness of the algorithm. Moreover, Round Robin has higher average turnaround time than other algorithms (such as SJF and SRT) that take process burst time into account and are designed to maximize throughput.

RR – Test Case 1 (Quantum = 2)

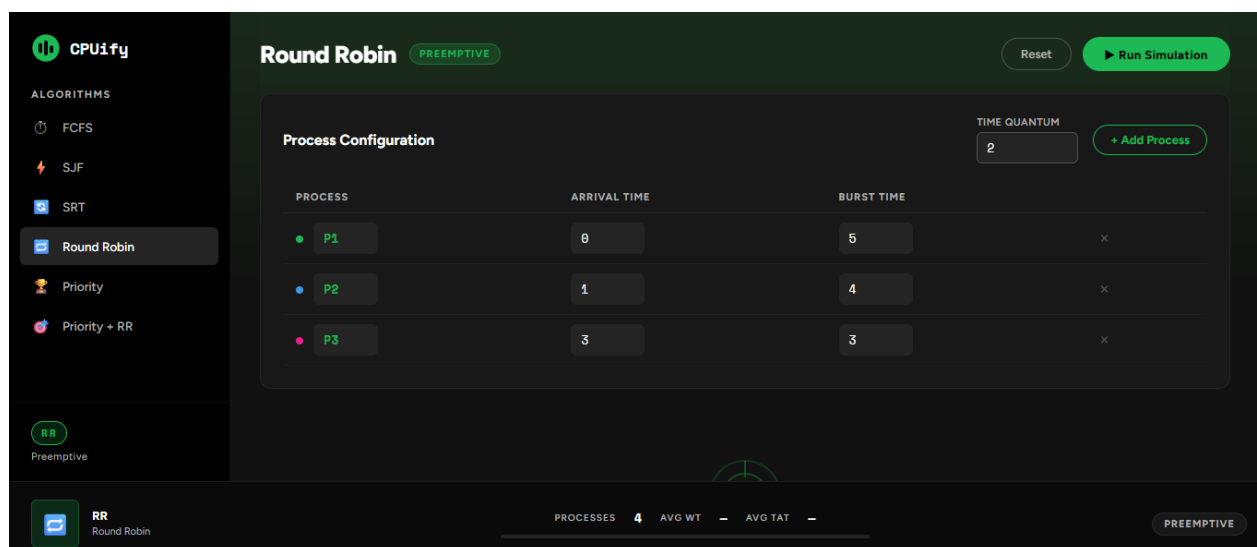


Figure 28. Interface of the System – RR

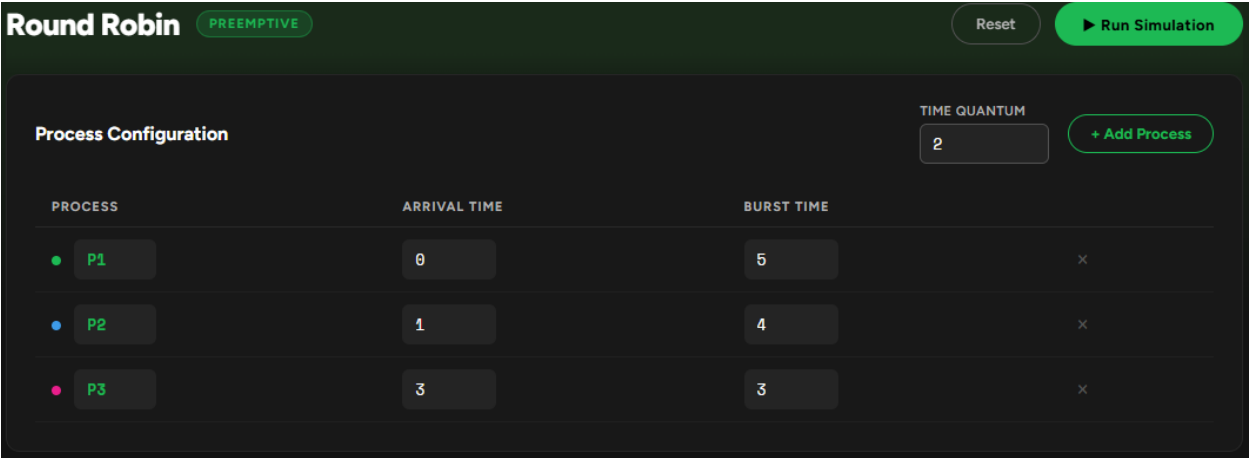


Figure 29. Input Parameters of RR Test Case 1

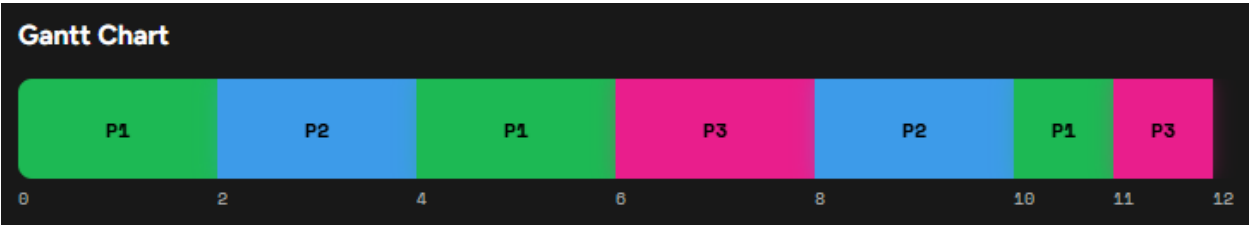


Figure 30. Gantt Chart of RR Test Case 1

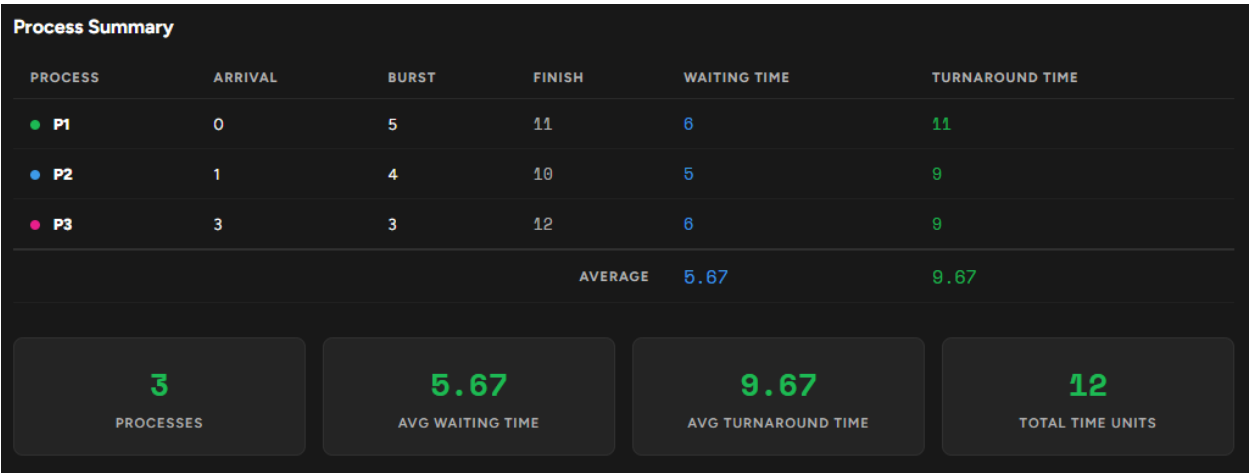


Figure 31. Process Summary of RR Test Case 1

At time $t=0$, only P1 is in the queue, so it begins execution for one time quantum, running from $t=0$ to $t=2$ and leaving a remaining time of 3. P2 arrives at $t=1$ and is added to the queue. By $t=2$, the queue becomes [P2, P1], allowing P2 to run from $t=2$ to $t=4$ with 2 units remaining. P3 arrives at $t=3$ and is placed at the end of the queue. At $t=4$, the queue is [P1, P3, P2], and P1 executes from $t=4$ to $t=6$, leaving 1 unit remaining. At $t=6$, the queue updates to [P3, P2, P1], and P3 runs from $t=6$ to $t=8$ with 1 unit remaining. At $t=8$, the queue becomes [P2, P1, P3], and P2 executes from $t=8$ to $t=10$, completing its execution. At $t=10$, the queue is [P1, P3], and P1 runs from $t=10$ to $t=11$, finishing its remaining burst. Finally, P3 executes from $t=11$ to $t=12$ and



completes. This sequence clearly demonstrates the cyclic nature of Round Robin scheduling, where CPU time is shared fairly and no single process dominates execution.

RR – Test Case 2 (Quantum = 3)

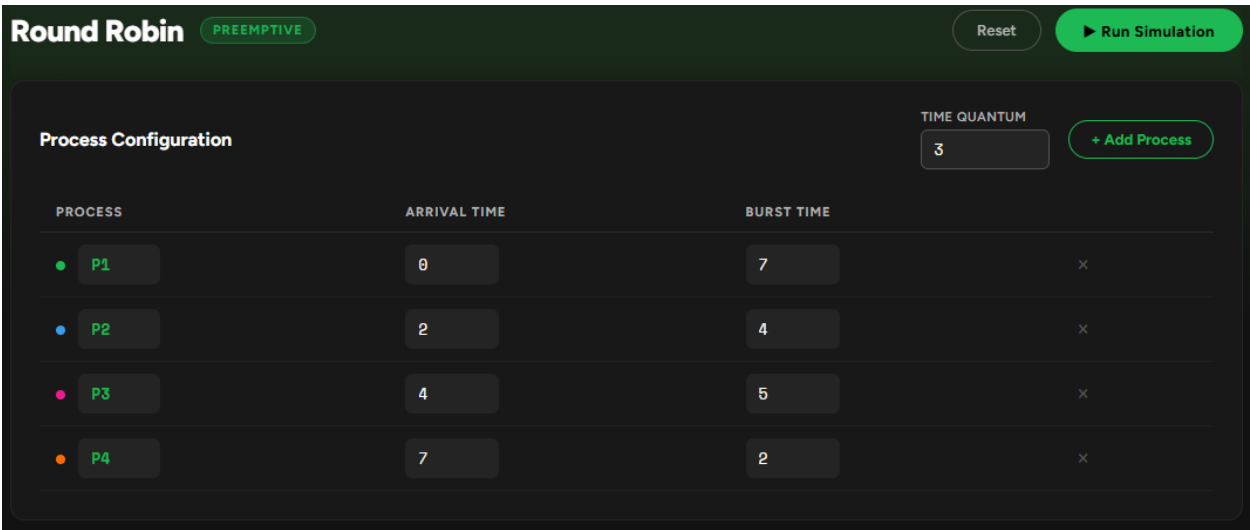
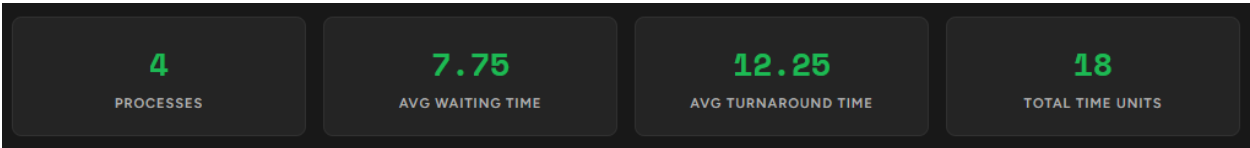


Figure 32. Input Parameters of RR Test Case 2



Figure 33. Gantt Chart of RR Test Case 2

Process Summary					
PROCESS	ARRIVAL	BURST	FINISH	WAITING TIME	TURNAROUND TIME
P1	0	7	16	9	16
P2	2	4	13	7	11
P3	4	5	18	9	14
P4	7	2	15	6	8
AVERAGE				7.75	12.25



Figures 34 and 35. Process Summary of RR Test Case 2



Execution begins with P1 running from $t=0$ to $t=3$, leaving a remaining time of 4. P2 arrives at $t=2$ and is added to the queue. At $t=3$, the queue becomes [P2, P1], allowing P2 to execute from $t=3$ to $t=6$ with 1 unit remaining. P3 arrives at $t=4$ and joins the queue. By $t=6$, the queue updates to [P1, P3, P2], and P1 runs again from $t=6$ to $t=9$, reducing its remaining time to 1. P4 arrives at $t=7$ and is placed at the end of the queue. At $t=9$, the queue becomes [P3, P2, P4, P1], and P3 executes from $t=9$ to $t=12$ with 2 units remaining. At $t=12$, the queue is [P2, P4, P1, P3], and P2 runs from $t=12$ to $t=13$, completing its execution. Next, P4 runs from $t=13$ to $t=15$ and finishes, followed by P1 from $t=15$ to $t=16$, and finally P3 from $t=16$ to $t=18$, completing all processes. This scenario shows that with a time quantum of 3, processes with larger burst times require multiple cycles to finish, resulting in higher waiting times compared to a smaller quantum such as 2.

5A. Priority Scheduling (Non-Preemptive)

Algorithm Description and Type: Non-Preemptive and lower number = higher priority

In non-preemptive Priority Scheduling, processes are given priorities upon input. The scheduler chooses the highest priority process from the ready queue (in this case, the process with the lowest numerical priority) when the CPU becomes free. This process then runs to completion without interruption. In the case of ties, the process with the earliest arrival time is selected to maintain determinism in the process execution order (Omar et al., 2021).

Advantages and Limitations

Advantages:

Priority Scheduling ensures that processes identified as critical or time-sensitive are given precedence in CPU allocation over less urgent tasks. It provides a clear and flexible mechanism for distinguishing process importance, allowing system designers to assign priorities based on specific requirements such as deadlines or system roles. Additionally, the non-preemptive version is relatively simple to implement and results in stable and predictable execution patterns, since once a process begins, it runs to completion without interruption.

Limitations:

Despite its benefits, Priority Scheduling can lead to starvation for processes with lower priority levels, especially when higher-priority tasks continue to arrive (Ali et al., 2021). The assignment of priorities is often subjective and may not always reflect the true urgency or importance of a process, which can result in inefficient scheduling decisions. Furthermore, without



the use of an aging mechanism that gradually increases the priority of waiting processes, the algorithm lacks a built-in solution to prevent starvation over time.

Priority (Non-Preemptive) Test Case 1

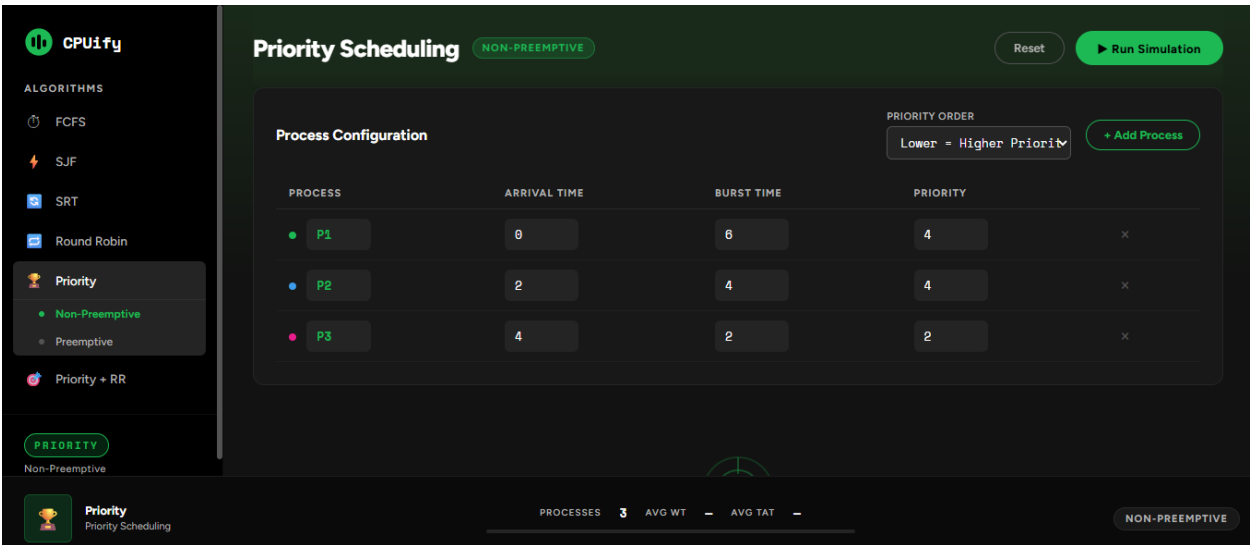


Figure 36. Interface of the System – Priority (Non-Preemptive)

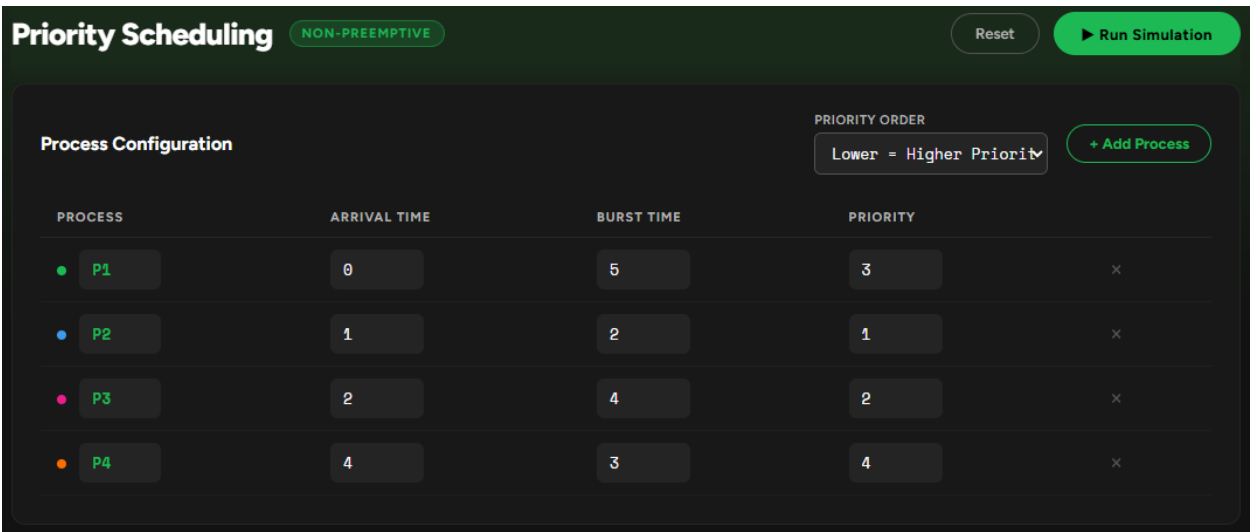


Figure 37. Input Parameters of Priority (Non-Preemptive) Test Case 1

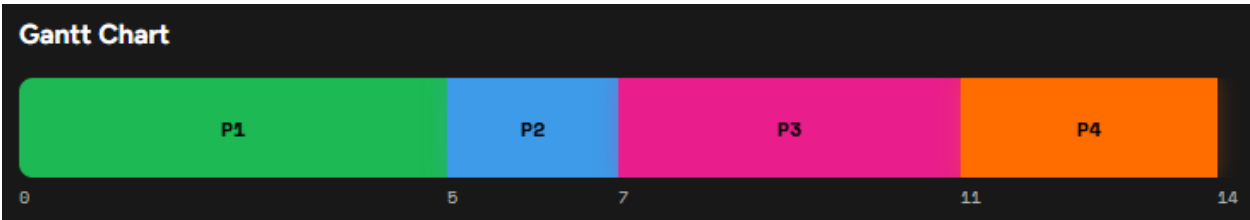


Figure 38. Gantt Chart of Priority (Non-Preemptive) Test Case 1



Process Summary					
PROCESS	ARRIVAL	BURST	FINISH	WAITING TIME	TURNAROUND TIME
P1	0	5	5	0	5
P2	1	2	7	4	6
P3	2	4	11	5	9
P4	4	3	14	7	10
AVERAGE				4.00	7.50

4 PROCESSES	4.00 AVG WAITING TIME	7.50 AVG TURNAROUND TIME	14 TOTAL TIME UNITS
----------------	--------------------------	-----------------------------	------------------------

Figures 39 and 40. Process Summary of Priority (Non-Preemptive) Test Case 1

At time $t=0$, P1 is the only available process, so it begins execution despite having a moderate priority level of 3 and completes at $t=5$. By the time the CPU becomes free, all remaining processes are already waiting in the queue: P2 with the highest priority (1), followed by P3 (2), and P4 with the lowest priority (4). The scheduler first selects P2, which runs from $t=5$ to $t=7$, followed by P3 from $t=7$ to $t=11$. Lastly, P4 executes from $t=11$ to $t=14$. Although P4 has a reasonable burst time, its low priority causes it to wait 7-time units before execution, demonstrating how priority-based scheduling can lead to starvation for lower-priority processes.

Priority (Non-Preemptive) Test Case 2

Priority Scheduling				NON-PREEMPTIVE	Reset	Run Simulation
Process Configuration				PRIORITY ORDER		
				Lower = Higher Priority		
				+ Add Process		
PROCESS	ARRIVAL TIME	BURST TIME	PRIORITY			
P1	0	4	4	x		
P2	0	3	2	x		
P3	0	6	1	x		
P4	0	2	3	x		

Figure 41. Input Parameters of Priority (Non-Preemptive) Test Case 2

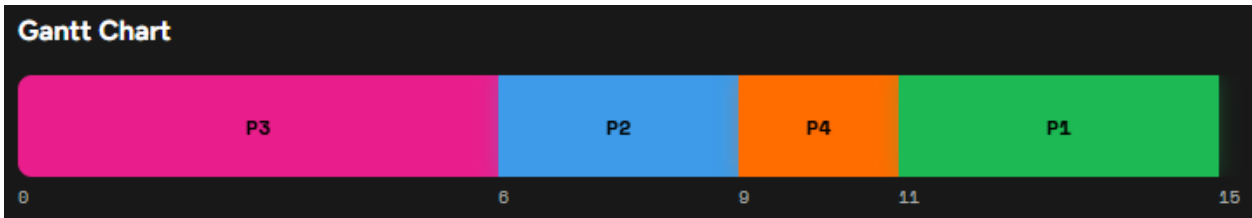
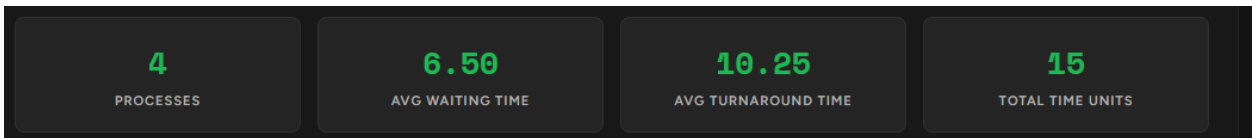


Figure 42. Gantt Chart of Priority (Non-Preemptive) Test Case 2

Process Summary					
PROCESS	ARRIVAL	BURST	FINISH	WAITING TIME	TURNAROUND TIME
P1	0	4	15	11	15
P2	0	3	9	6	9
P3	0	6	6	0	6
P4	0	2	11	9	11
AVERAGE				6.50	10.25



Figures 43 & 44. Process Summary of Priority (Non-Preemptive) Test Case 2

All four processes arrive at the same time ($t=0$), making the scheduling decision based entirely on priority rather than arrival order. The scheduler arranges them as follows: P3 (priority 1), P2 (priority 2), P4 (priority 3), and P1 (priority 4). Execution proceeds with P3 running from $t=0$ to $t=6$, followed by P2 from $t=6$ to $t=9$, then P4 from $t=9$ to $t=11$, and finally P1 from $t=11$ to $t=15$. Even though P1 arrived first, it is executed last due to having the lowest priority, clearly illustrating how priority-based scheduling overrides arrival order and can significantly delay lower-priority processes.

5. Priority Scheduling (Preemptive)

Algorithm Description and Type: Preemptive and lower number = higher priority

Preemptive Priority Scheduling is an extension of the non-preemptive variant where the CPU can be taken away from a currently running process if a newly arriving process has a higher priority. At each time unit, the scheduler re-evaluates all available processes and assigns the CPU to the one with the highest priority. If a new process arrives with a higher priority than the process currently executing, the running process is immediately preempted and returned to the ready queue (Ali et al., 2021). Within the simulation, priority direction is user-configurable: lower numerical



value indicates higher priority by default, though this can be reversed through the interface. Ties in priority are broken by arrival time to ensure deterministic scheduling behavior.

Advantages and Limitations

Advantages:

Preemptive Priority Scheduling provides faster response for high-priority processes, as they are not forced to wait for a lower-priority process to complete before receiving CPU time. This makes the algorithm well-suited for real-time systems and environments where certain tasks must be handled urgently. It also improves CPU utilization by ensuring that the most critical process is always executing rather than waiting idle behind a lower-priority one. Compared to the non-preemptive version, it more closely reflects the behavior of real operating system schedulers, which continuously re-evaluate process urgency as the system state changes.

Limitations:

The preemptive variant introduces more frequent context switching compared to the non-preemptive version, as the CPU may be reassigned at every time unit when a higher-priority process arrives. This overhead can reduce overall system efficiency in environments with heavy process loads. Like its non-preemptive counterpart, it remains highly susceptible to starvation: low-priority processes may be indefinitely delayed if a continuous stream of higher-priority processes keeps arriving. Additionally, the accuracy and fairness of the algorithm depend entirely on the quality of the assigned priorities, which may not always reflect the actual urgency or importance of each process in dynamic workloads (Ali et al., 2021).

Priority (Preemptive) - Test Case 1

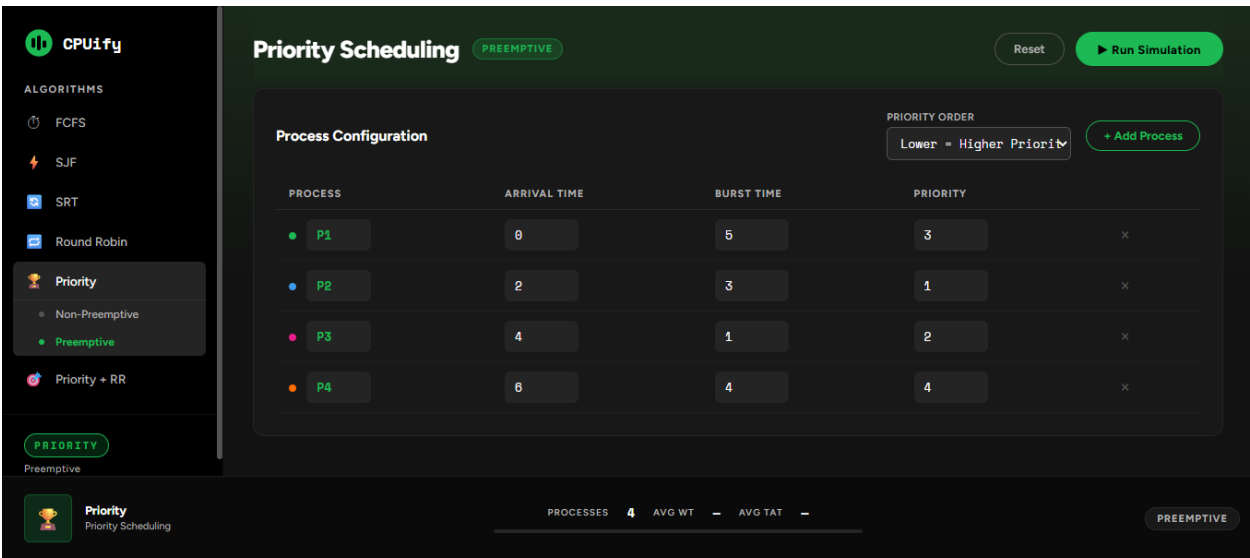


Figure 45. Interface of the System – Priority (Preemptive)

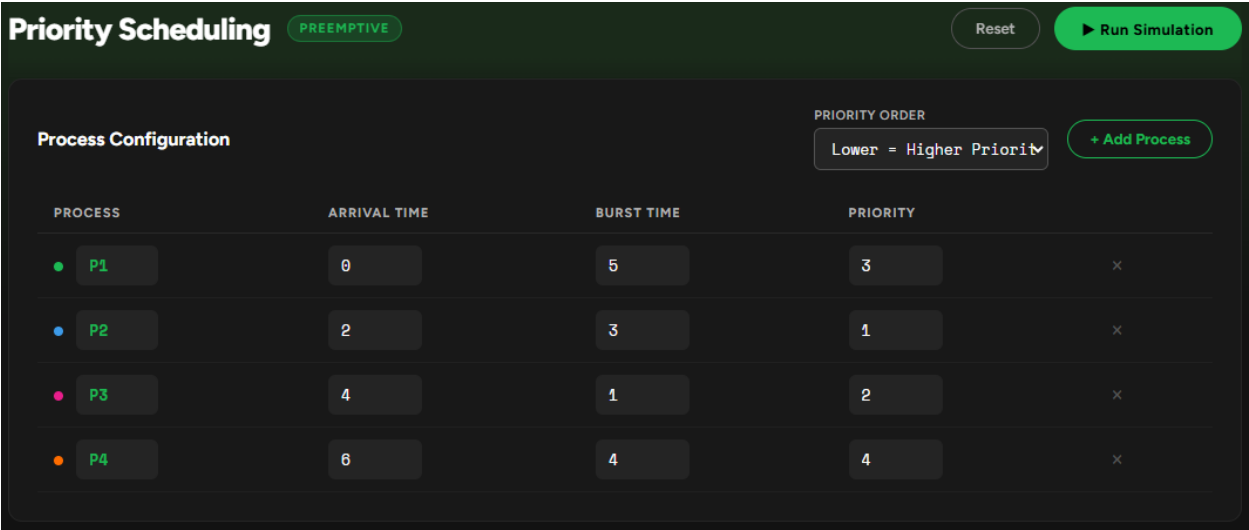


Figure 46. Input Parameters of Priority (Preemptive) Test Case 1

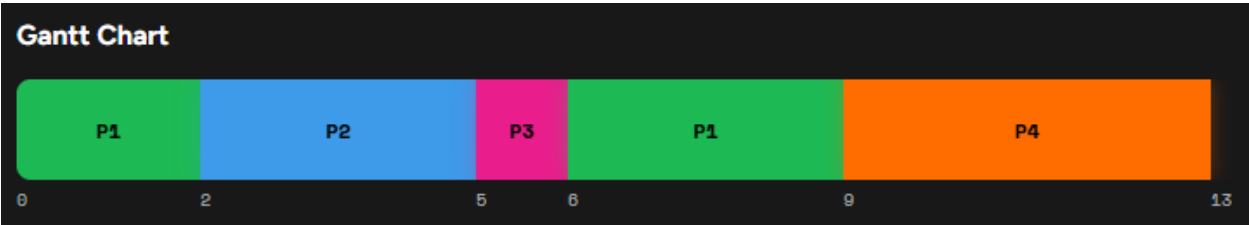
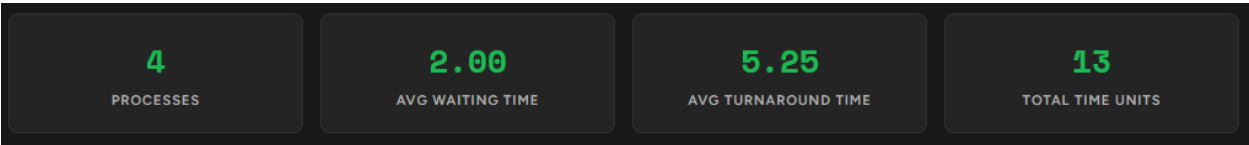


Figure 47. Gantt Chart of Priority (Preemptive) Test Case 1

Process Summary					
PROCESS	ARRIVAL	BURST	FINISH	WAITING TIME	TURNAROUND TIME
P1	0	5	9	4	9
P2	2	3	5	0	3
P3	4	1	6	1	2
P4	6	4	13	3	7
AVERAGE				2.00	5.25



Figures 48 & 49. Process Summary of Priority (Preemptive) Test Case 1

At time $t=0$, P1 is the only available process and immediately begins execution with a burst time of 5 and priority 3. At $t=2$, P2 arrives with priority 1, which is the highest priority in the set, so P1 is immediately preempted and P2 takes over the CPU. P2 runs uninterrupted from $t=2$ to $t=5$, completing its full burst of 3-time units ($FT=5$, $WT=0$, $TAT=3$). At $t=4$, P3 arrives with priority 2, which is lower than P2's priority 1, so no preemption occurs and P3 waits in the ready queue. Once P2 finishes at $t=5$, the scheduler selects P3 (priority 2) over P1 (priority 3), and P3



executes from $t=5$ to $t=6$ ($FT=6$, $WT=1$, $TAT=2$). At $t=6$, $P4$ arrives with priority 4, the lowest in the set, while $P1$ (priority 3) is still waiting. The scheduler resumes $P1$, which runs from $t=6$ to $t=9$ completing its remaining 3 units ($FT=9$, $WT=4$, $TAT=9$). Finally, $P4$ executes last from $t=9$ to $t=13$ ($FT=13$, $WT=3$, $TAT=7$). This test case demonstrates how a single high-priority arrival can preempt a running process mid-execution, significantly reducing the waiting time of urgent processes at the cost of delaying lower-priority ones.

Priority (Preemptive) - Test Case 2

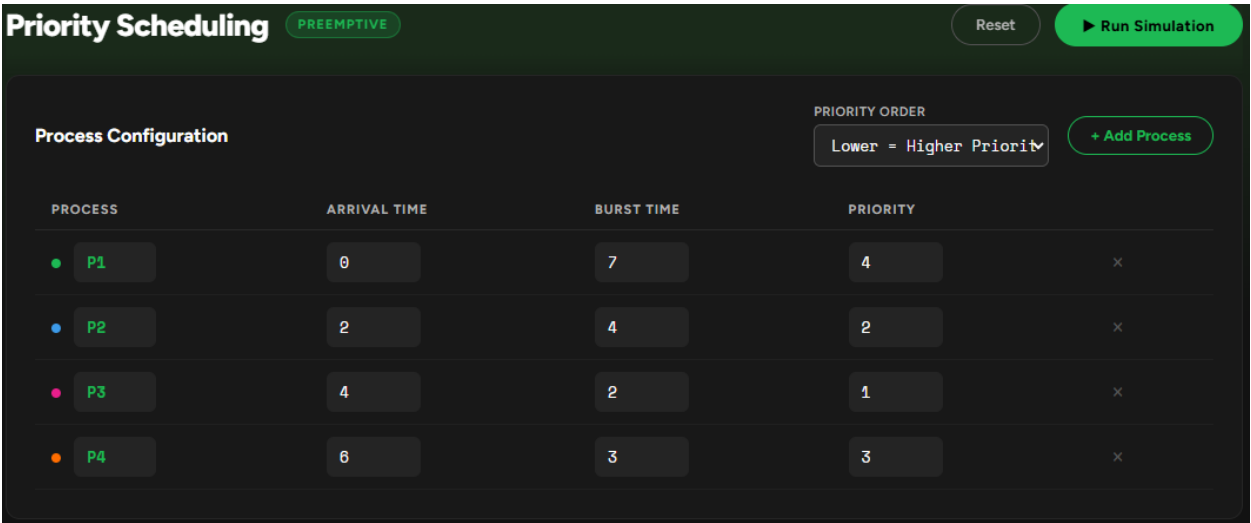
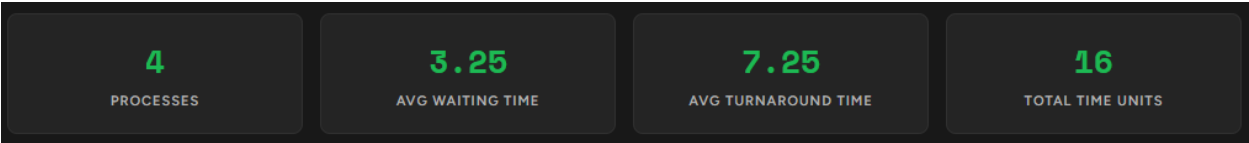


Figure 50. Input Parameters of Priority (Preemptive) Test Case 2



Figure 51. Gantt Chart of Priority (Preemptive) Test Case 2

Process Summary					
PROCESS	ARRIVAL	BURST	FINISH	WAITING TIME	TURNAROUND TIME
P1	0	7	16	9	16
P2	2	4	8	2	6
P3	4	2	6	0	2
P4	6	3	11	2	5
AVERAGE				3.25	7.25



Figures 52 & 53. Process Summary of Priority (Preemptive) Test Case 2

At time $t=0$, P1 is the only available process and begins execution with a burst time of 7 and priority 4, the lowest priority in the set. At $t=2$, P2 arrives with priority 2, which is higher than P1's priority 4, so P1 is immediately preempted after running only 2 units and P2 takes over the CPU. P2 runs from $t=2$ to $t=4$, when P3 arrives with priority 1, the highest priority in the set, immediately preempting P2. P3 executes uninterrupted from $t=4$ to $t=6$, completing its full burst ($FT=6$, $WT=0$, $TAT=2$). Once P3 finishes, P2 resumes with 2 remaining units and runs from $t=6$ to $t=8$ ($FT=8$, $WT=2$, $TAT=6$). At $t=6$, P4 had arrived with priority 3, which is lower than P2's priority 2, so it could not preempt and had to wait. After P2 completes at $t=8$, P4 is selected next due to its priority 3 over P1's priority 4 and runs from $t=8$ to $t=11$ ($FT=11$, $WT=2$, $TAT=5$). Finally, P1 resumes with 5 remaining units and executes from $t=11$ to $t=16$ ($FT=16$, $WT=9$, $TAT=16$). This test case clearly illustrates how low-priority processes accumulate significant waiting time through cascading preemptions, as P1 was interrupted twice and forced to wait a total of 9-time units before completing which is a direct consequence of the strict priority enforcement in preemptive scheduling.

6. Priority Scheduling with Round Robin

Algorithm Description and Type: Preemptive

Priority Scheduling with Round Robin is a hybrid CPU scheduling method that combines priority-based ordering with the fairness of time-sliced execution. In this approach, processes are first grouped according to their priority levels, and the scheduler always selects the highest-priority group for execution. Within that group, processes are handled using Round Robin, where each process receives CPU time based on a fixed time quantum. Once all processes in the current priority level are completed, the scheduler moves on to the next lower priority group and repeats the same cycle (Ali et al., 2021).

Within the simulation, both the time quantum and priority rules are defined by the user before execution begins. During scheduling, the ready queue is organized based on priority, and Round Robin rotation is applied only among processes within the highest active priority level. This design closely reflects multilevel queue scheduling used in modern operating systems, where processes are separated by urgency and managed using combined scheduling strategies.

Advantages and Limitations



Advantages:

Priority Scheduling with Round Robin prevents starvation within the same priority level by ensuring that all processes in that group receive CPU time in a rotating manner based on a fixed quantum. It effectively combines the urgency-based decision-making of Priority Scheduling with the fairness and time-sharing characteristics of Round Robin, resulting in a more balanced scheduling strategy. Because of this hybrid structure, it also provides a more realistic approximation of how modern operating system schedulers operate compared to single-criterion algorithms (Abuazoum, 2025).

Limitations:

Even with its advantages, this hybrid approach is more complex to implement than simpler scheduling algorithms, especially when processes with different priorities arrive dynamically during execution. Its performance is also highly dependent on the chosen time quantum, which requires careful adjustment to achieve optimal results. Additionally, processes in lower-priority groups may still experience long waiting times if higher-priority groups contain a large amount of total burst time, limiting overall responsiveness for less urgent tasks.

Priority with Round Robin Test Case 1

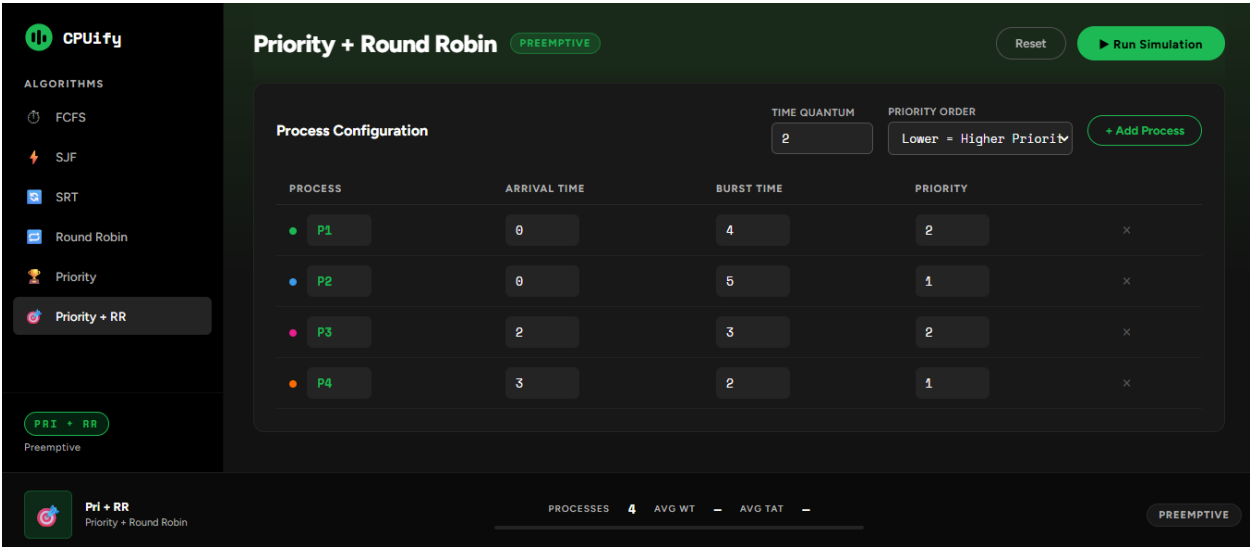


Figure 54. Interface of the System – Priority with Round Robin



Priority + Round Robin

PREEMPTIVE

Reset

Run Simulation

Process Configuration

TIME QUANTUM

2

PRIORITY ORDER

Lower = Higher Priority

+ Add Process

PROCESS	ARRIVAL TIME	BURST TIME	PRIORITY	
P1	0	4	2	×
P2	0	5	1	×
P3	2	3	2	×
P4	3	2	1	×

Figure 55. Input Parameters of PRR Test Case 1

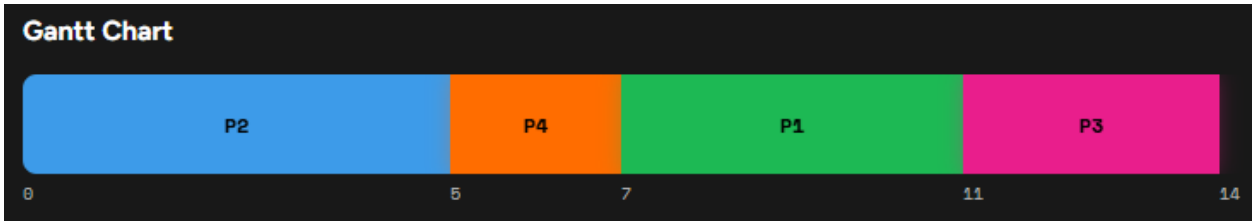


Figure 56. Gantt Chart of PRR Test Case 1

Process Summary					
PROCESS	ARRIVAL	BURST	FINISH	WAITING TIME	TURNAROUND TIME
P1	0	4	11	7	11
P2	0	5	5	0	5
P3	2	3	14	9	12
P4	3	2	7	2	4
AVERAGE				4.50	8.00

4

PROCESSES

4.50

AVG WAITING TIME

8.00

AVG TURNAROUND TIME

14

TOTAL TIME UNITS

Figures 57 & 58. Process Summary of PRR Test Case 1

At t=0, both P1 (priority 2) and P2 (priority 1) arrive, but P2 is selected first because lower numbers indicate higher priority. P2 executes from t=0 to t=2, leaving 3 burst units remaining, while P3 arrives at t=2 and joins the queue. P2 continues again from t=2 to t=4 since it still holds priority-1 status, reducing its remaining burst to 1. At t=3, P4 arrives with priority 1 and joins the same high-priority group. By t=4, the priority-1 queue contains P2 (rem=1) and P4 (rem=2), while P1 and P3 remain in priority 2. P2 finishes first from t=4 to t=5 (FT=5, WT=0, TAT=5), followed



by P4 from $t=5$ to $t=7$ ($FT=7$, $WT=2$, $TAT=4$). After all priority-1 processes complete, the scheduler moves to priority-2 processes using Round Robin, where P1 runs from $t=7$ to $t=9$ (remaining burst $4 \rightarrow 2$), then P3 runs from $t=9$ to $t=11$ ($3 \rightarrow 1$), followed by P1 again from $t=11$ to $t=13$ to complete ($FT=13$, $WT=7$, $TAT=11$), and finally P3 finishes from $t=13$ to $t=14$ ($FT=14$, $WT=9$, $TAT=12$). Overall, the priority scheduling clearly favors P2 and P4, which complete early with minimal waiting times, while P1 and P3 experience significantly longer delays because they only begin execution after all higher-priority processes are done.

Priority with Round Robin Test Case 2

Priority + Round Robin

PREEMPTIVE

Reset

Run Simulation

Process Configuration

TIME QUANTUM

3

PRIORITY ORDER

Lower = Higher Priority

Add Process

PROCESS	ARRIVAL TIME	BURST TIME	PRIORITY	
P1	0	6	2	×
P2	0	4	1	×
P3	3	5	2	×
P4	4	3	1	×

Figure 59. Input Parameters of PRR Test Case 2

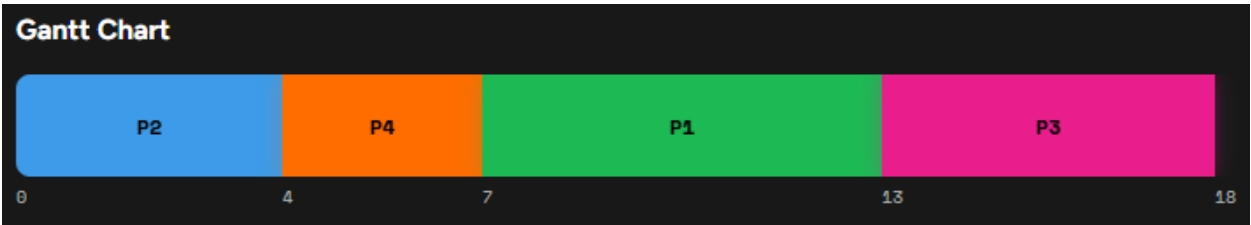
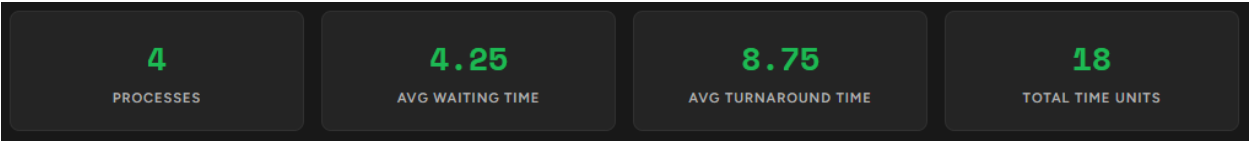


Figure 60. Gantt Chart of PRR Test Case 2

Process Summary					
PROCESS	ARRIVAL	BURST	FINISH	WAITING TIME	TURNAROUND TIME
P1	0	6	13	7	13
P2	0	4	4	0	4
P3	3	5	18	10	15
P4	4	3	7	0	3
AVERAGE				4.25	8.75



Figures 61 & 62. Process Summary of PRR Test Case 2

At $t=0$, P1 and P2 have both arrived, and since P2 has higher priority (priority 1 vs P1's priority 2), P2 is selected and runs its first quantum from $t=0$ to $t=3$, leaving 1 unit of burst remaining. During this time, P3 arrives at $t=3$ with priority 2 and joins the queue. At $t=3$, P2 (priority 1, $rem=1$) still outranks both P1 and P3, so it continues and completes its remaining execution from $t=3$ to $t=4$ ($FT=4$, $WT=0$, $TAT=4$). At $t=4$, P4 arrives with priority 1 and immediately takes the highest-priority slot, running from $t=4$ to $t=7$ and finishing ($FT=7$, $WT=0$, $TAT=3$). With all priority-1 processes completed, the scheduler moves to priority-2 processes P1 ($rem=6$) and P3 ($rem=5$), using Round Robin. P1 runs first from $t=7$ to $t=10$ ($6 \rightarrow 3$), then P3 runs from $t=10$ to $t=13$ ($5 \rightarrow 2$), followed by P1 again from $t=13$ to $t=16$ ($3 \rightarrow 0$) completing execution ($FT=16$, $WT=7$, $TAT=16$), and finally P3 finishes from $t=16$ to $t=18$ ($2 \rightarrow 0$) ($FT=18$, $WT=10$, $TAT=15$). The separation between priority levels is clear, as both priority-1 processes finish early with zero waiting time, while P1 and P3 only begin after all higher-priority work is done, resulting in significantly higher waiting times.

7. Comparison of Results Across Algorithms

The table below summarizes the average performance metrics gathered from all test cases in this simulation, offering a single, consistent basis for evaluating and comparing the behavior of each scheduling algorithm under different workload conditions. By combining results from multiple scenarios, it highlights how each algorithm performs in terms of average waiting time and turnaround time, while also revealing patterns such as consistency, responsiveness, and sensitivity to factors like arrival order, burst time variation, and time quantum.

Metric	FCFS	SJF	SRT	Round Robin	Priority NP	Priority P	Priority + RR
Type	Non-Preemptive	Non-Preemptive	Preemptive	Preemptive	Non-Preemptive	Preemptive	Preemptive
TC1 Avg. WT	5.75	3.25	2.00	3.00	3.25	2.00	4.50
TC1 Avg. TAT	8.75	6.25	5.00	6.00	6.00	5.25	8.00



TC2 Avg. WT	6.75	3.50	4.25	5.50	5.00	3.25	4.25
TC2 Avg. TAT	9.75	6.50	7.75	9.25	8.75	7.25	8.50
Overall Avg. WT	6.25	3.38	3.13	4.25	4.13	2.63	4.38
Overall Avg. TAT	9.25	6.38	6.38	7.63	7.38	6.25	8.25
Best Use Case	Batch/simple systems	Low- variance workloads	Short-burst heavy systems	Interactive/time- sharing	Urgency- based systems	Real-time systems	OS-like mixed workloads

Table . Comparison of Results Across Algorithms

Analysis of Results:

FCFS showed the highest average waiting time among all seven algorithms, with overall values of 6.25 (AWT) and 9.25 (ATAT). The difference between its two test cases, 5.75 in Test Case 1 and 6.75 in Test Case 2, highlights how strongly its performance depends on arrival order. In Test Case 2, P1 used the CPU continuously for 9 time units, forcing P2, P3, and P4 to wait regardless of their burst times. P4, even with the shortest burst of 1 unit, waited 10 time units before execution, illustrating the convoy effect and showing why FCFS is inefficient when burst times vary greatly.

SJF performed significantly better, with an overall average waiting time of 3.38 and turnaround time of 6.38. Its results were also stable across both test cases, with waiting times of 3.25 and 3.50, showing the benefit of always selecting the shortest available job. In Test Case 1, shorter processes like P4 and P2 were executed earlier after P1, reducing total waiting time compared to FCFS. However, Test Case 2 revealed its weakness, as P2 waited 8 time units despite arriving early, since shorter jobs kept being selected first.

SRT produced the best result in Test Case 1, matching Priority Preemptive with an average waiting time of 2.00 and a turnaround time of 5.00. Its advantage comes from allowing interruptions when shorter jobs arrive, reducing waiting time for those processes. Using the same data as SJF Test Case 1, SRT lowered the average waiting time from 3.25 to 2.00 by letting P2 interrupt P1 at t=2. In Test Case 2, however, its average waiting time increased to 4.25 because P1 was interrupted multiple times, leading to a total waiting time of 10 units before completion at t=18.



Round Robin recorded an average waiting time of 3.00 in Test Case 1 with a quantum of 2 and 5.50 in Test Case 2 with a quantum of 3, resulting in an overall average of 4.25. The higher waiting time in Test Case 2 shows the impact of a larger quantum, where shorter jobs had to wait longer between turns. Although its waiting times are not the lowest, Round Robin ensured that all processes received CPU time regularly, preventing starvation in both test cases.

Non-preemptive Priority Scheduling achieved an overall average waiting time of 4.13, placing it between SJF and Round Robin. In Test Case 1, it performed well with a waiting time of 3.25 due to clear priority differences. In Test Case 2, where all processes arrived at the same time, execution strictly followed priority order. This caused the lowest-priority process, P1, to wait until $t=11$, resulting in a waiting time of 11 units despite arriving first. This demonstrates that without interruption, lower-priority processes can experience long delays.

Preemptive Priority Scheduling achieved the lowest overall average waiting time of 2.63 and an average turnaround time of 6.25. In Test Case 1, immediate interruption allowed the highest-priority process to finish with zero waiting time, lowering the average to 2.00. In Test Case 2, multiple interruptions caused P1 to wait 9 time units, while P3 completed with no delay. The algorithm performs well in handling urgent tasks but creates large differences in waiting times, increasing the risk of starvation for lower-priority processes.

Priority Scheduling with Round Robin produced the most consistent results, with average waiting times of 4.50 and 4.25 and an overall average of 4.38. Unlike pure priority scheduling, it prevents any process within the same priority level from dominating the CPU. In Test Case 1, higher-priority processes completed first, while lower-priority ones shared the CPU through rotation. In Test Case 2, staggered arrivals allowed smoother transitions between priority levels, slightly improving performance. Although it recorded the highest average turnaround time of 8.25 due to added overhead, it is the only approach that balances both priority and fairness effectively.



III. Summary

Key Insights Gained from Implementation

The implementation of the six CPU scheduling algorithms offered insights beyond theoretical knowledge. It illustrated the behavior of each algorithm under various workloads, and it exposed the considerations in scheduler design.

- **No single scheduling algorithm is universally optimal.** All algorithms trade-off fairness, efficiency and simplicity in different ways, there is no ‘best’ scheduling algorithm. For example, SRT has the shortest average waiting time but causes a large overhead for context-switching and has the potential to starve longer processes. On the other hand, FCFS is easy to implement but is inefficient with mixed process lengths and so is poor in mixed workloads.
- **The Convoy Effect of FCFS is evident.** If a long process arrives before shorter processes, all following processes are delayed, whether they are small or large. This illustrates the potential of first-come-first-serve to adversely affect system throughput.
- **SJF and SRT require knowledge of or information about CPU bursts, which is not realistic in actual operating systems.** In real-world scenarios, a scheduler estimates burst times using predictive models like exponential averaging (based on past CPU usage) to estimate the next CPU burst time.
- **The time quantum in Round Robin impacts system performance.** If the quantum is very short, then there will be a lot of context switching and overhead. If the quantum is very long, then the algorithm will behave like FCFS. This demonstrates that performance is sensitive to the choice of quantum, depending on the distribution of burst times.
- **A more practical scheduling algorithm is a combination of Priority Scheduling and Round Robin.** While it’s harder to implement, it is a good model for modern operating systems like Windows and Linux, which handle different classes of processes through a combination of priorities and fairness within each class.



Observed Differences in Performance

Aspect	FCFS	SJF / SRT	Round Robin (RR)	Priority Scheduling	Priority + Round Robin
Fairness	Low fairness; earlier processes dominate CPU time	Moderate fairness; favors shorter jobs	High fairness; equal CPU sharing per cycle	Low fairness; favors high-priority processes	Moderate to high fairness; fair within same priority levels
Efficiency	Low efficiency in mixed workloads	Very high efficiency; SRT is the best overall	Moderate efficiency; affected by quantum size	Variable efficiency; depends on priority distribution	Moderate efficiency; better than pure Priority but lower than SRT
Response Time	Poor when long job arrives first	Excellent; very fast for short processes	Good; frequent CPU access improves responsiveness	Good for high-priority tasks, poor for others	Good for high-priority processes; improved fairness within groups
Starvation Risk	None	High (SJF/SRT)	None	High without aging	Reduced compared to Priority alone, but still possible across priority levels
Overall Behavior	Simple but inefficient for diverse workloads	Best for minimizing waiting time but less practical	Best for interactive systems	Best for urgency-based systems	Most balanced hybrid; realistic OS-like scheduling behavior

The table compares six CPU scheduling algorithms based on various performance metrics including fairness, efficiency, response time, starvation and overall behavior. It shows that although FCFS is simple, it is inefficient and unfair in mixed workload scenarios. Shortest Job First (SJF) and Shortest Remaining Time (SRT) have the highest efficiency but are prone to



starvation as they prioritize shorter processes. Round Robin has the best fairness and is suitable for interactive systems, but efficiency is heavily influenced by the time quantum. Priority Scheduling provides greater control by servicing important processes but can lead to starvation. On the other hand, Priority Scheduling with Round Robin provides a more balanced solution by combining priority-based scheduling with time-sharing within priority levels, which is more realistic of operating system schedulers. In conclusion, the performance comparison highlights that there are trade-offs associated with each algorithm, and no one solution is best.

Practical Relevance in Modern Operating Systems

Today's operating systems do not implement a pure form of scheduling algorithms. Instead, they implement hybrid and adaptive approaches. For instance, the Linux kernel employs the Completely Fair Scheduler (CFS) that keeps track of each process's virtual run-time and arranges them in a red-black tree. It always picks the process with the smallest amount of CPU time, coming close to an ideal fair scheduling model for all processes.

Likewise, Microsoft Windows uses a multilevel feedback queue (MLFQ) with several priority levels and priority boosts. This is similar to a Priority Scheduling with Round Robin, considering both priority and sharing time.

On the other hand, real-time embedded systems, such as FreeRTOS and VxWorks, often employ fixed-priority preemptive scheduling (Oliveira et al., 2020). This is similar to Priority Scheduling, but with extra measures and restrictions to ensure safe execution of critical tasks without destabilizing the system and causing starvation in safety-critical systems.

In summary, the benefits and drawbacks of these basic scheduling algorithms need to be understood as they provide the foundation for understanding, setting up and tuning real-world schedulers used in different types of computer systems.

Challenges Encountered

The development of the scheduling simulator posed several challenges in terms of accuracy, consistency and readability of the different algorithms. A major problem was encountered when producing Gantt charts for the Shortest Remaining Time (SRT) algorithm. Due to SRT performing scheduling decisions at each time unit, it initially resulted in many fragmented segments, each of which is one time unit long. To overcome this, we had to merge consecutive time slots of the same process, thereby improving the visual interpretation.



The other issue was the queue management in Priority Scheduling with Round Robin. This involved managing several queues for different priority levels and ensuring that new processes were placed in the correct queue, and preempted processes were added back to the correct queue. This required state maintenance at each time unit to ensure consistency and avoid scheduling problems.

Coping with special cases was also important. Cases in which multiple processes had the same arrival time and burst length required a deterministic tie breaking rule. This allowed the system to generate the same results for identical test cases, even when the simulation was run multiple times.

Finally, the tick marks on the Gantt chart were misaligned, displaying wrong values like 0, 3, 11, 15, 20, instead of the correct values 0, 2, 3, 11, 15, 20. This happened because the first tick was rendered with an offset caused by the border of the chart, but the other ticks were not, causing the time markers to be misaligned and some marks to be missing. Also, the real-time Gantt chart was another design consideration. Given the varying total execution times of different test cases (ranging from short to long times), it was crucial to introduce dynamic scaling. This required the computation of different unit lengths to ensure readability of the Gantt chart, regardless of the total duration of the simulation.



IV. References

- [1].Chakraborty, P. (2024). Operating systems: evolutionary concepts and modern design principles (First). CRC Press. <https://doi.org/10.1201/9781003383055>
- [2].Omar, H. K., Jihad, K. H., & Hussein, S. F. (2021). Comparative analysis of the essential CPU scheduling algorithms. Bulletin of Electrical Engineering and Informatics, 10(5), 2742–2750. <https://doi.org/10.11591/eei.v10i5.2812>
- [3].Stallings, W. (2018). Operating systems. internals and design principles. <http://library.megu.edu.ua:8180/jspui/handle/123456789/4147>
- [4].Oliveira, G., & Lima, G. (2020). Evaluation of Scheduling Algorithms for Embedded FreeRTOS-based Systems. IEEE Xplore, 1–8. <https://doi.org/10.1109/sbesc51047.2020.9277851>
- [5].Ali, S. M., Alshahrani, R. F., Hadadi, A. H., Alghamdi, T. A., Almuhsin, F. H., & El-Sharawy, E. E. (2021, January 1). A review on the CPU Scheduling Algorithms: Comparative Study. 교보문고 스콜라. <https://scholar.kyobobook.co.kr/article/detail/4040046871079>
- [6].Abuazoum, S. (2025). A comparative analysis of VXWorks OS and Windows CE in embedded Real-Time applications. Wadi Alshatti University Journal of Pure and Applied Sciences, 9–15. https://doi.org/10.63318/waujpasv3i2_02



V. Repository Link

<https://github.com/rjrualles/OS-CASESTUDY.git>



VI. Source Code

The system is built using a structured collection of source files that define its interface, styling, and core functionality. These files work together to form a complete CPU scheduling simulator that allows users to input processes, select scheduling algorithms, and visualize execution results.

The implementation integrates frontend layout, styling, and JavaScript logic to ensure smooth interaction and dynamic behavior. The user interface components are designed to respond in real time, updating outputs such as the Gantt chart and results table based on user inputs and selected algorithms.

Overall, the source code is organized in a modular and readable format to support maintainability and ease of understanding. Each part of the system is clearly separated according to its function, making it easier to trace how inputs are processed and how results are generated.

A. UI and Input Handling

This section includes the core functions responsible for managing user interaction within the application. These functions handle tasks such as changing algorithms, adding or removing process rows, collecting and validating input data, executing simulations, and resetting the interface. Together, they ensure smooth interaction between the user and the system.

```
function switchAlgo() {
  const a = ALGOS[currentAlgo];

  // Update topbar heading and preemptive badge
  algoTitle.textContent = a.name;
  algoBadge.textContent = a.type;

  // Update sidebar footer
  document.getElementById('sidebar-algo-name').textContent = a.short;
  document.getElementById('sidebar-algo-type').textContent = a.type;

  // Update the Spotify-style now-playing bar at the bottom
  document.getElementById('np-algo-icon').textContent = a.icon;
  document.getElementById('np-algo-name').textContent = a.short;
  document.getElementById('np-algo-full').textContent = a.name;
  document.getElementById('np-type').textContent = a.type;
  document.getElementById('np-cover').firstElementChild.textContent = a.icon;

  // Show the Time Quantum field only for algorithms that slice CPU time
  quantumGroup.style.display = a.quantum ? 'flex' : 'none';

  // Show the Priority order selector only for Priority-based algorithms
  priorityGroup.style.display = a.priority ? 'flex' : 'none';

  // Show/hide the Priority column header in the process input table
```



```
colPriority.style.display = a.priority ? " : 'none';

// Show/hide Priority input cells in every existing process row
document.querySelectorAll('.priority-cell').forEach(cell => {
  cell.style.display = a.priority ? " : 'none';
});
}
```

Figure 63. switchAlgo() function

Figure 63 shows that this function updates the entire interface whenever the user selects a different scheduling algorithm. It retrieves the selected algorithm’s configuration and refreshes the title, type badge, and sidebar details. It also controls visibility of optional inputs such as Time Quantum and Priority settings and dynamically shows or hides the Priority column across all existing rows, so the UI always matches the selected algorithm without reloading.

```
function addProcessRow() {
  processCount++;
  const idx = processRows.length;      // used to cycle through the color palette
  const color = COLORS[idx % COLORS.length]; // wrap palette if > 10 processes
  const pid = `P${processCount}`;
  const tr = document.createElement('tr');

  tr.innerHTML = `
    <td>
      <span class="pid-dot" style="background:${color}"></span>
      <input class="pid-input" type="text" value="${pid}" maxlength="6" placeholder="P1"/>
    </td>
    <td><input type="number" value="${Math.floor(Math.random() * 4)}" min="0" max="99"
placeholder="0"/></td>
    <td><input type="number" value="${1 + Math.floor(Math.random() * 8)}" min="1" max="99"
placeholder="1"/></td>
    <td class="priority-cell" style="display:${ALGOS[currentAlgo].priority ? " : 'none'}">
      <input type="number" value="${1 + Math.floor(Math.random() * 5)}" min="1" max="99" placeholder="1"/>
    </td>
    <td><button class="btn-remove" title="Remove">x</button></td>
  `;

  // Remove button: deletes the row from the DOM and drops it from the tracking array
  tr.querySelector('.btn-remove').addEventListener('click', () => {
    tr.remove();
    processRows = processRows.filter(r => r.tr !== tr);
  });

  tbody.appendChild(tr);
  processRows.push({ tr, color }); // register so color can be retrieved during rendering
}
```

Figure 64. addProcessRow() function

In this figure, it demonstrates that this function creates and adds a new process entry in the input table. It generates a unique process ID, assigns a color from a rotating palette, and inserts input fields for arrival time, burst time, and optional priority. Each row includes a remove button that deletes the row and updates the tracking array. The row and its color are stored to maintain consistent visualization in the Gantt chart.

```
function resetAll() {
  tbody.innerHTML = ""; // remove all existing process rows from DOM
}
```



```
processRows = []; // clear the color tracking array
processCount = 0; // reset auto-naming counter
hideOutput();
// Re-add the minimum required 3 processes
addProcessRow();
addProcessRow();
addProcessRow();
}
```

Figure 65. *resetAll()* function

In this example, this function resets the simulator to its default state by clearing all process rows, resetting counters and stored data, and hiding any displayed results. It then reinitializes the table by adding three new process rows, ensuring the system meets the minimum requirement and starts from a clean configuration.

```
function hideOutput() {
  outputPanel.style.display = 'none';
  emptyState.style.display = "";
  document.getElementById('np-avg-wt').textContent = '-';
  document.getElementById('np-avg-tat').textContent = '-';
  document.getElementById('progress-bar').style.width = '0%';
}
```

Figure 66. *hideOutput()* function

Figure 66 shows that this function hides the output section, including the Gantt chart and results table, and displays the empty placeholder screen. It also resets the average waiting time, turnaround time, and progress bar to default values, preventing outdated results from appearing after a reset or algorithm change.

```
function readProcesses() {
  const rows = tbody.querySelectorAll('tr');
  const processes = [];

  rows.forEach((tr, i) => {
    const inputs = tr.querySelectorAll('input');
    const pid = inputs[0].value.trim() || `P${i + 1}`; // default if blank
    const at = Math.max(0, parseInt(inputs[1].value) || 0); // arrival time ≥ 0
    const bt = Math.max(1, parseInt(inputs[2].value) || 1); // burst time ≥ 1
    const prio = ALGOS[currentAlgo].priority
      ? (parseInt(inputs[3].value) || 1) // priority (when applicable)
      : 0; // 0 = unused

    processes.push({
      pid,
      at,
      bt,
      prio,
      color: processRows[i] ? processRows[i].color : COLORS[i % COLORS.length]
    });
  });

  return processes;
}
```

Figure 67. *readProcesses()* function



In this instance, this function reads all user inputs from the table and converts them into a structured array of process objects. It validates values by enforcing minimum limits, assigns default values where necessary, and includes priority only when required by the selected algorithm. Each process object also carries its assigned color for consistent rendering.

```
function runSimulation() {
  const processes = readProcesses();

  // Enforce minimum of 3 processes per case study specification
  if (processes.length < 3) {
    alert('Please add at least 3 processes (minimum required by the case study).');
    return;
  }

  // Dispatch to the appropriate scheduling algorithm.
  // Each algorithm returns a timeline: [{pid, start, end, color}, ...]
  let timeline = [];

  switch (currentAlgo) {
    case 'fcfs':
      timeline = fcfs(processes);
      break;
    case 'sjf':
      timeline = sjf(processes);
      break;
    case 'srt':
      timeline = srt(processes);
      break;
    case 'rr':
      // Read time quantum from input; default to 2 if blank or invalid
      timeline = roundRobin(processes, parseInt(quantumInput.value) || 2);
      break;
    case 'priority':
      timeline = priorityScheduling(processes);
      break;
    case 'priority_rr':
      timeline = priorityRR(processes, parseInt(quantumInput.value) || 2);
      break;
  }

  // Derive per-process Waiting Time and Turnaround Time from the timeline
  const results = computeResults(processes, timeline);

  // Render all three output sections
  renderGantt(timeline);
  renderResults(results);
  updateNowPlaying(results);

  // Show the output panel and scroll it into view smoothly
  outputPanel.style.display = "";
  emptyState.style.display = 'none';
  outputPanel.scrollToView({ behavior: 'smooth', block: 'nearest' });
}
```

Figure 68. runSimulation() function

In this example, it demonstrates that this function serves as the main controller of the application. It collects and validates process data, ensures the minimum number of processes is met, and selects the appropriate scheduling algorithm based on user choice. After generating the



execution timeline, it computes performance metrics and triggers rendering of the Gantt chart, results table, and status bar before displaying the final output.

B. Scheduling Algorithms

All six scheduling functions follow a consistent structure: each takes a list of processes (and a time quantum when required) and produces a finalized execution timeline. The timeline is composed of segments defined by process ID, start time, end time, and color, with idle periods explicitly labeled using 'idle'. Before returning the result, every function refines its raw timeline by passing it through `mergeTimeline()` to combine consecutive segments and ensure a cleaner, more readable output.

```
function fcfs(processes) {  
  // Sort by arrival time; preserve original insertion order for equal arrivals  
  const sorted = [...processes].sort((a, b) => a.at - b.at);  
  const timeline = [];  
  let time = 0; // CPU clock - tracks the current time  
  
  for (const p of sorted) {  
    // If the next process hasn't arrived yet, CPU sits idle  
    if (time < p.at) {  
      timeline.push({ pid: 'idle', start: time, end: p.at, color: null });  
      time = p.at; // jump clock forward to the process's arrival  
    }  
  
    // Execute this process fully - no preemption possible in FCFS  
    timeline.push({ pid: p.pid, start: time, end: time + p.bt, color: p.color });  
    time += p.bt; // advance clock by the full burst duration  
  }  
  
  return mergeTimeline(timeline);  
}
```

Figure 69. FCFS Scheduling Algorithm Function

Figure 69 demonstrates that this function implements First-Come, First-Served scheduling by sorting processes based on arrival time and executing them sequentially without interruption. If the CPU becomes idle before the next process arrives, an idle segment is inserted. While simple, this method can lead to long waiting times when early processes have large burst durations.

```
function sjf(processes) {  
  // Deep-copy processes; add 'done' flag to track which have completed  
  const jobs = processes.map(p => ({ ...p, done: false }));  
  const timeline = [];  
  let time = 0;  
  let completed = 0;  
  const n = jobs.length;  
  
  while (completed < n) {  
    // Build the available pool: arrived and not yet finished  
    const avail = jobs.filter(j => !j.done && j.at <= time);  
  
    if (avail.length === 0) {  
      // No process ready - CPU idles until the earliest next arrival  
      const nextAt = Math.min(...jobs.filter(j => !j.done).map(j => j.at));  
      timeline.push({ pid: 'idle', start: time, end: nextAt, color: null });  
      time = nextAt;  
    }  
  
    // Find the process with the shortest burst time  
    const shortest = avail.reduce((min, j) => j.bt < min.bt ? j : min);  
    timeline.push({ pid: shortest.pid, start: time, end: time + shortest.bt, color: shortest.color });  
    time += shortest.bt;  
    completed++;  
    shortest.done = true;  
  }  
  
  return mergeTimeline(timeline);  
}
```



```
    timeline.push({ pid: 'idle', start: time, end: nextAt, color: null });
    time = nextAt;
    continue;
  }

  // Select the process with the shortest burst; break ties by arrival time
  avail.sort((a, b) => a.bt - b.bt || a.at - b.at);
  const p = avail[0];

  // Execute fully (non-preemptive) - no interruptions allowed
  timeline.push({ pid: p.pid, start: time, end: time + p.bt, color: p.color });
  time += p.bt;
  p.done = true;
  completed++;
}

return mergeTimeline(timeline);
}
```

Figure 70. SJF Scheduling Algorithm Function

As shown here, this function applies Shortest Job First scheduling by repeatedly selecting the shortest available process and executing it to completion. It checks which processes have arrived, sorts them by burst time, and runs the smallest one. If no process is ready, the timeline advances with an idle segment. Although efficient, longer processes may experience starvation.

```
function srt(processes) {
  // Deep-copy with remaining time (rem) and done flag
  const jobs = processes.map(p => ({ ...p, rem: p.bt, done: false }));
  const timeline = [];
  let time = 0;
  let completed = 0;
  const n = jobs.length;

  while (completed < n) {
    // All processes that have arrived and still need CPU time
    const avail = jobs.filter(j => !j.done && j.at <= time);

    if (avail.length === 0) {
      // No process available - fast-forward to next arrival
      const nextAt = Math.min(...jobs.filter(j => !j.done).map(j => j.at));
      timeline.push({ pid: 'idle', start: time, end: nextAt, color: null });
      time = nextAt;
      continue;
    }

    // Pick process with least remaining time; ties broken by arrival time
    avail.sort((a, b) => a.rem - b.rem || a.at - b.at);
    const p = avail[0];
    const last = timeline[timeline.length - 1];

    // Extend the previous segment if the same process continues uninterrupted;
    // otherwise push a new 1-unit segment (indicates a preemption or fresh start)
    if (last && last.pid === p.pid && last.end === time) {
      last.end = time + 1; // extend - no preemption occurred this unit
    } else {
      timeline.push({ pid: p.pid, start: time, end: time + 1, color: p.color });
    }

    p.rem--; // consume 1 time unit of this process's burst
    time++;
  }
}
```



```
// Check if this process has now used all of its requested CPU time
if (p.rem === 0) {
  p.done = true;
  completed++;
}
}

return mergeTimeline(timeline);
}
```

Figure 71. SRT Scheduling Algorithm Function

In this example, it shows that this function implements Shortest Remaining Time scheduling, a preemptive version of SJF. It evaluates processes at every time unit and always runs the one with the smallest remaining burst time. If a shorter job arrives, the current process is interrupted. Execution segments are built incrementally and later merged for clarity.

```
function roundRobin(processes, quantum) {
  // Sort jobs by arrival time to enqueue them in the correct arrival order
  const jobs = processes.map(p => ({ ...p, rem: p.bt, done: false }))
  .sort((a, b) => a.at - b.at);

  const timeline = [];
  const queue = []; // FIFO ready queue
  let time = 0;
  let idx = 0; // index into sorted jobs - marks the next unqueued process

  // If no process has arrived yet at t=0, fast-forward the clock to the
  // first arrival and insert an idle segment. Without this guard, when all
  // processes have AT > 0, the queue starts empty and the while(queue>0) loop
  // never executes, producing a completely empty timeline.
  if (idx < jobs.length && jobs[idx].at > time) {
    timeline.push({ pid: 'idle', start: time, end: jobs[idx].at, color: null });
    time = jobs[idx].at;
  }

  // Seed the queue with all processes that have now arrived
  while (idx < jobs.length && jobs[idx].at <= time) {
    queue.push(jobs[idx++]);
  }

  while (queue.length > 0) {
    const p = queue.shift(); // dequeue the next process
    const run = Math.min(p.rem, quantum); // execute for at most one quantum

    // Record this execution slice in the timeline
    timeline.push({ pid: p.pid, start: time, end: time + run, color: p.color });
    time += run;
    p.rem -= run;

    // Enqueue any processes that arrived while this slice was running
    while (idx < jobs.length && jobs[idx].at <= time) {
      queue.push(jobs[idx++]);
    }

    if (p.rem > 0) {
      // Process has remaining work - re-enqueue at the back (preempted)
      queue.push(p);
    } else {
      p.done = true; // process has finished all of its burst time
    }
  }
}
```



```
// If queue is now empty but unstarted processes exist, idle until next arrival
if (queue.length === 0 && idx < jobs.length) {
  timeline.push({ pid: 'idle', start: time, end: jobs[idx].at, color: null });
  time = jobs[idx].at;
  // Enqueue processes that have now arrived
  while (idx < jobs.length && jobs[idx].at <= time) {
    queue.push(jobs[idx++]);
  }
}
}

return mergeTimeline(timeline);
}
```

Figure 72. Round Robin Scheduling Algorithm Function

In this instance, it presents that this function uses Round Robin scheduling, where processes are handled in a queue and each receives a fixed time slice. If a process is not completed within its quantum, it is reinserted into the queue. The function also handles newly arriving processes and idle periods, ensuring fairness and preventing starvation.

```
function priorityScheduling(processes) {
  const order = priorityOrder.value; // 'lower' or 'higher' from UI dropdown
  const jobs = processes.map(p => ({ ...p, done: false }));
  const timeline = [];
  let time = 0;
  let completed = 0;
  const n = jobs.length;

  while (completed < n) {
    // Collect all processes that have arrived and are not yet done
    const avail = jobs.filter(j => !j.done && j.at <= time);

    if (avail.length === 0) {
      // No process ready - idle until the earliest remaining arrival
      const nextAt = Math.min(...jobs.filter(j => !j.done).map(j => j.at));
      timeline.push({ pid: 'idle', start: time, end: nextAt, color: null });
      time = nextAt;
      continue;
    }

    // Sort by priority according to the user-selected interpretation,
    // then by arrival time to break ties deterministically
    avail.sort((a, b) =>
      order === 'lower'
        ? a.prio - b.prio || a.at - b.at // lower number wins
        : b.prio - a.prio || a.at - b.at // higher number wins
    );

    const p = avail[0]; // highest-priority available process

    // Run to full completion - non-preemptive, no interruptions
    timeline.push({ pid: p.pid, start: time, end: time + p.bt, color: p.color });
    time += p.bt;
    p.done = true;
    completed++;
  }

  return mergeTimeline(timeline);
}
```

Figure 73. Priority Scheduling Algorithm Function (Non-Preemptive)



Figure 73 shows that this function executes processes based on priority, selecting the highest-priority process available and running it to completion. The priority direction can be configured by the user. Since it is non-preemptive, lower-priority processes may be delayed indefinitely if higher-priority tasks continue to arrive.

```
function priorityPreemptive(processes) {
  const order = priorityOrder.value; // 'lower' or 'higher' from UI dropdown
  const jobs = processes.map(p => ({ ...p, rem: p.bt, done: false }));
  const timeline = [];
  let time = 0;
  let completed = 0;
  const n = jobs.length;

  while (completed < n) {
    // All processes that have arrived and still need CPU time
    const avail = jobs.filter(j => !j.done && j.at <= time);

    if (avail.length === 0) {
      // No process ready — fast-forward to next arrival
      const nextAt = Math.min(...jobs.filter(j => !j.done).map(j => j.at));
      timeline.push({ pid: 'idle', start: time, end: nextAt, color: null });
      time = nextAt;
      continue;
    }

    // Select highest-priority available process (same sort as non-preemptive)
    avail.sort((a, b) =>
      order === 'lower'
        ? a.prio - b.prio || a.at - b.at // lower number = higher priority
        : b.prio - a.prio || a.at - b.at // higher number = higher priority
    );

    const p = avail[0];
    const last = timeline[timeline.length - 1];

    // Extend last segment if same process continues; new segment if preempted
    if (last && last.pid === p.pid && last.end === time) {
      last.end = time + 1; // same process — extend, no preemption this unit
    } else {
      timeline.push({ pid: p.pid, start: time, end: time + 1, color: p.color });
    }

    p.rem--;
    time++;

    if (p.rem === 0) {
      p.done = true;
      completed++;
    }
  }

  return mergeTimeline(timeline);
}
```

Figure 74. Priority Scheduling Algorithm Function (Preemptive)

In this example, this function implements preemptive priority scheduling by assigning the CPU at each time unit to the highest-priority available process. It selects and runs the top process for one unit, immediately preempting it if a higher-priority process arrives. Consecutive executions



of the same process are merged to keep the timeline clean, while idle periods are added when no process is ready. Although responsive to high-priority tasks, this method can cause starvation for lower-priority processes.

```
function priorityRR(processes, quantum) {
  const order = priorityOrder.value; // 'lower' or 'higher' from UI dropdown

  // Sort by arrival time so we can enqueue in the correct order
  const jobs = processes.map(p => ({ ...p, rem: p.bt, done: false }))
    .sort((a, b) => a.at - b.at);

  const timeline = [];
  const queue = []; // unified ready queue - re-sorted by priority each round
  let time = 0;
  let idx = 0; // index into sorted jobs for arrival tracking
  let completed = 0;

  if (idx < jobs.length && jobs[idx].at > time) {
    timeline.push({ pid: 'idle', start: time, end: jobs[idx].at, color: null });
    time = jobs[idx].at;
  }

  // Seed queue with all processes that have now arrived
  while (idx < jobs.length && jobs[idx].at <= time) {
    queue.push(jobs[idx++]);
  }

  while (completed < jobs.length) {
    if (queue.length === 0) {
      // Nothing ready - fast-forward to the next process arrival
      const nextAt = Math.min(...jobs.filter(j => !j.done).map(j => j.at));
      timeline.push({ pid: 'idle', start: time, end: nextAt, color: null });
      time = nextAt;
      while (idx < jobs.length && jobs[idx].at <= time) {
        queue.push(jobs[idx++]);
      }
      continue;
    }

    // Re-sort the ready queue by priority before each scheduling decision.
    // This ensures a newly arrived high-priority process gets precedence
    // over processes already in the queue from previous rounds.
    queue.sort((a, b) =>
      order === 'lower'
        ? a.prio - b.prio || a.at - b.at
        : b.prio - a.prio || a.at - b.at
    );

    const p = queue.shift(); // highest-priority process in queue
    const run = Math.min(p.rem, quantum); // execute for at most one quantum

    timeline.push({ pid: p.pid, start: time, end: time + run, color: p.color });
    time += run;
    p.rem -= run;

    // Enqueue processes that arrived during this execution slice
    while (idx < jobs.length && jobs[idx].at <= time) {
      queue.push(jobs[idx++]);
    }

    if (p.rem > 0) {
      // Unfinished - re-enqueue; will compete with new arrivals on next sort
    }
  }
}
```




```
        queue.push(p);
    } else {
        p.done = true;
        completed++;
    }
}

return mergeTimeline(timeline);
}
```

Figure 75. Priority Scheduling with Round Robin Algorithm Function

As shown here, this function combines priority scheduling with Round Robin. The ready queue is sorted by priority before each execution, ensuring higher-priority processes are selected first, while processes with equal priority share CPU time using a time quantum. This approach balances responsiveness and fairness.

C. Timeline Utility

This section contains a helper function used to improve the structure of the execution timeline. It processes the raw output generated by scheduling algorithms and simplifies it by merging consecutive segments of the same process, resulting in a more organized and visually clear representation.

```
function mergeTimeline(tl) {
    if (tl.length === 0) return [];
    const merged = [ ...tl[0] ]; // begin with a fresh copy of the first segment

    for (let i = 1; i < tl.length; i++) {
        const last = merged[merged.length - 1];
        // Merge condition: same PID and segments are contiguous (no time gap)
        if (tl[i].pid === last.pid && tl[i].start === last.end) {
            last.end = tl[i].end; // extend the existing segment's end time
        } else {
            merged.push( ...tl[i] ); // different process or gap - start a new segment
        }
    }

    return merged;
}
```

Figure 76. mergeTimeline() function

In this instance, it demonstrates that this utility function simplifies the execution timeline by merging consecutive segments belonging to the same process, provided they are continuous. It reduces unnecessary fragmentation in the Gantt chart while preserving separate segments for interrupted executions.

D. Result Computation

This section focuses on calculating performance metrics based on the completed execution timeline. It derives essential values such as finish time, turnaround time, and waiting time for each process, ensuring accurate evaluation of the scheduling algorithm’s performance.


```
function computeResults(processes, timeline) {  
  // Build a finish-time map: pid → last end time seen in the timeline  
  const finish = {};  
  for (const seg of timeline) {  
    if (seg.pid !== 'idle') {  
      finish[seg.pid] = seg.end; // overwrite on each occurrence; last = actual finish  
    }  
  }  
  
  return processes.map(p => {  
    const ft = finish[p.pid] ?? (p.at + p.bt); // finish time (fallback for edge cases)  
    const tat = ft - p.at; // turnaround time  
    const wt = Math.max(0, tat - p.bt); // waiting time (never negative)  
    return { ...p, ft, tat, wt };  
  });  
}
```

Figure 77. computeResults() function

Figure 77 shows that this function calculates key performance metrics for each process using the execution timeline. It determines finish time from the last occurrence in the timeline, computes turnaround time as the difference between finish and arrival time, and calculates waiting time as turnaround minus burst time, ensuring values remain valid.

E. Rendering

This section is responsible for displaying the simulation results in the user interface. It includes functions that generate the Gantt chart, populate the results table, and update summary statistics, transforming computed data into a clear and visually informative format.

```
function renderGantt(timeline) {  
  // Clear any previous chart output  
  ganttChart.innerHTML = "";  
  ganttTimeline.innerHTML = "";  
  if (timeline.length === 0) return;  
  
  const totalTime = timeline[timeline.length - 1].end; // total simulation length  
  const MIN_BLOCK = 28; // must match .gantt-block { min-width } in CSS  
  
  // unitPx: how many pixels represent one time unit.  
  // Clamped so the chart is always readable across different timeline lengths.  
  const unitPx = Math.max(30, Math.min(80, Math.floor(700 / totalTime)));  
  
  // — Pre-compute the ACTUAL rendered pixel width of every block  
  // This is the critical step: CSS enforces min-width: 28px on .gantt-block,  
  // so a 1-unit block at unitPx=30 renders as 30px - but a very short block  
  // could be forced wider by min-width. We must use the SAME widths for both  
  // the block divs AND the tick marks, otherwise ticks drift out of alignment.  
  const blockWidths = timeline.map(seg => {  
    const dur = seg.end - seg.start;  
    return Math.max(MIN_BLOCK, dur * unitPx); // mirror the CSS min-width rule  
  });  
  
  // — Row 1: Gantt process blocks  
  timeline.forEach((seg, i) => {  
    const dur = seg.end - seg.start; // duration of this execution segment  
    const w = blockWidths[i]; // actual rendered pixel width
```



```
const div = document.createElement('div');
div.className = 'gantt-block' + (seg.pid === 'idle' ? ' idle' : '');
div.style.width = w + 'px';
div.style.minWidth = w + 'px'; // override CSS min-width with exact value

// Stagger the CSS entry animation so blocks appear left-to-right
div.style.animationDelay = (i * 0.04) + 's';

if (seg.pid !== 'idle') {
  div.style.background = seg.color;
  div.style.boxShadow = `0 0 12px ${seg.color}55`; // subtle glow effect
}

// Tooltip: hover to see exact time range and duration of this segment
div.title = `${seg.pid === 'idle' ? 'Idle' : seg.pid}: t=${seg.start} → t=${seg.end} (${dur} unit${dur !== 1 ? 's' : ''})`;
div.textContent = seg.pid === 'idle' ? '' : seg.pid;

ganttChart.appendChild(div);
});

// — Row 2: Timeline tick marks
// Build a cumulative position map: time value → exact pixel offset from
// the left edge of the Gantt chart. Computed by summing actual block widths
// (same Math.max(MIN_BLOCK, dur*unitPx) values used for the block divs).
//
// Example: FCFS [P1 BT=2, P2 BT=1, P3 BT=8, P4 BT=4, P5 BT=5] at unitPx=35:
// blockWidths = [70, 35, 280, 140, 175]
// posMap = { 0:0, 2:70, 3:105, 11:385, 15:525, 20:700 }
//
// Tick labels are positioned ABSOLUTELY at posMap[t] from the left.
// This avoids the flow-div overlap bug where t=0 renders as 0px width,
// causing t=2's label to stack on top of t=0 and disappear.

// Build cumulative position map
const posMap = {};
let cumPx = 0;
timeline.forEach((seg, i) => {
  posMap[seg.start] = cumPx;
  cumPx += blockWidths[i];
});
posMap[timeline[timeline.length - 1].end] = cumPx; // final right edge

// Make the timeline container relatively positioned so absolute children work
ganttTimeline.style.position = 'relative';
ganttTimeline.style.height = '16px';
ganttTimeline.style.width = cumPx + 'px';

// Collect all unique boundary times and render one absolutely-positioned
// label per boundary - each label's left edge sits exactly at posMap[t] px
const boundaries = [...new Set(timeline.flatMap(s => [s.start, s.end]))]
  .sort((a, b) => a - b);

boundaries.forEach(t => {
  const tickDiv = document.createElement('div');
  tickDiv.className = 'gantt-tick';
  tickDiv.style.position = 'absolute';
  tickDiv.style.left = posMap[t] + 'px'; // exact pixel alignment
  tickDiv.style.whiteSpace = 'nowrap';
  tickDiv.textContent = t;
  gantTimeline.appendChild(tickDiv);
});
}
```



Figure 78. *renderGantt()* function

In this example, it shows that this function generates the visual Gantt chart by creating proportional blocks for each execution segment. It calculates pixel widths based on total simulation time, ensures minimum visibility, and aligns time markers accurately using cumulative positioning to maintain correct spacing.

```
function renderResults(results) {
  resultTbody.innerHTML = "";
  let totalWT = 0, totalTAT = 0;

  // Render one row per process with all computed timing values
  results.forEach(r => {
    totalWT += r.wt;
    totalTAT += r.tat;

    const tr = document.createElement('tr');
    tr.innerHTML = `
      <td>
        <span class="pid-dot" style="background:${r.color}"></span>
        <strong>${r.pid}</strong>
      </td>
      <td>${r.at}</td>
      <td>${r.bt}</td>
      <td class="finish-cell">${r.ft}</td>
      <td class="wt-cell">${r.wt}</td>
      <td class="tat-cell">${r.tat}</td>
    `;
    resultTbody.appendChild(tr);
  });

  // Compute averages to 2 decimal places for display
  const n = results.length;
  const avgWT = (totalWT / n).toFixed(2);
  const avgTAT = (totalTAT / n).toFixed(2);

  // Append the average summary row at the bottom of the process table
  const avgRow = document.createElement('tr');
  avgRow.style.borderTop = '2px solid rgba(255,255,255,0.1)';
  avgRow.innerHTML = `
    <td colspan="4" style="text-align:right; font-weight:700; color:var(--sub);
      font-size:11px; letter-spacing:1px; text-transform:uppercase;">Average</td>
    <td class="wt-cell" style="font-size:15px;">${avgWT}</td>
    <td class="tat-cell" style="font-size:15px;">${avgTAT}</td>
  `;
  resultTbody.appendChild(avgRow);

  // Render the four summary stat cards
  const totalSimTime = Math.max(...results.map(r => r.ft));
  statsRow.innerHTML = `
    <div class="stat-card">
      <span class="stat-val">${n}</span>
      <span class="stat-label">Processes</span>
    </div>
    <div class="stat-card">
      <span class="stat-val">${avgWT}</span>
      <span class="stat-label">Avg Waiting Time</span>
    </div>
    <div class="stat-card">
      <span class="stat-val">${avgTAT}</span>
      <span class="stat-label">Avg Turnaround Time</span>
  `;
}
```



```
</div>
<div class="stat-card">
  <span class="stat-val">${totalSimTime}</span>
  <span class="stat-label">Total Time Units</span>
</div>
`;
}
```

Figure 79. renderResults() function

As shown here, this function displays the computed results in a table, listing each process with its timing values. It calculates total and average waiting and turnaround times, appends an average row, and updates summary statistics such as total processes and total execution time.

```
function updateNowPlaying(results) {
  const n = results.length;
  const totalWT = results.reduce((s, r) => s + r.wt, 0);
  const totalTAT = results.reduce((s, r) => s + r.tat, 0);
  const avgWT = (totalWT / n).toFixed(2);
  const avgTAT = (totalTAT / n).toFixed(2);

  document.getElementById('np-processes').textContent = n;
  document.getElementById('np-avg-wt').textContent = avgWT;
  document.getElementById('np-avg-tat').textContent = avgTAT;

  // Animate progress bar to full width to signal simulation is complete
  document.getElementById('progress-bar').style.width = '100%';
}
```

Figure 80. updateNowPlaying() function

Figure 80 demonstrates that this function updates the status bar with real-time simulation statistics, including process count and average waiting and turnaround times. It also fills the progress bar to indicate that the simulation has completed successfully.

F. Initialization

This section handles the initial setup of the application when it first loads. It prepares the default state by creating sample process entries, assigning predefined values, and applying the initial interface configuration so the system is ready for immediate use.

```
function init() {
  // Add 3 default process rows (minimum required by the case study)
  addProcessRow();
  addProcessRow();
  addProcessRow();

  // Overwrite random defaults with a clear, predictable first-run example
  const rows = tbody.querySelectorAll('tr');
  const defaults = [
    [0, 6], // P1: arrives at 0, needs 6 units
    [2, 4], // P2: arrives at 2, needs 4 units
    [4, 2], // P3: arrives at 4, needs 2 units
  ];

  rows.forEach((tr, i) => {
    const inputs = tr.querySelectorAll('input');
    if (defaults[i]) {
```



```
inputs[1].value = defaults[i][0]; // set arrival time
inputs[2].value = defaults[i][1]; // set burst time
}
});

// Apply the default FCFS UI (no quantum field, no priority column)
switchAlgo();
}

// Start the application
init();
```

Figure 81. init() function

In this instance, this function initializes the application on page load by creating three default process rows and assigning preset values for a sample scenario. It then applies the default algorithm layout, ensuring the interface is ready for immediate interaction.